CXC: Chest X-ray Classifier

Joshan John

3092883

Submitted in partial fulfillment for the degree of

B.Sc. in Computing Science

Griffith College Dublin

June, 2026

Under the supervision of Jennifer Lebron

Disclaimer

I hereby certify that this material, which I now submit for assessment on the programme of study leading to the Degree of Bachelor of Science in Computing at Griffith College Dublin, is entirely my own work and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

Signed: Joshan John **Date: 01/06/2026**

Acknowledgements

I express my sincere gratitude to Jennifer Lebrone, who was my supervisor & computing program director at Griffith College Dublin, for her invaluable guidance, continuous support, and constructive feedback throughout the completion of this work. Her knowledge, encouragement, and academic insight have been instrumental in the successful development of this project. I also like to extend my appreciation to Griffith College Dublin for providing an excellent academic environment and the necessary resources to support my learning and research.

I am especially and sincerely grateful to Dr. Ryan Roby for his exceptional guidance, continuous encouragement, and unwavering support throughout the development of this Chest X-ray Classifier CNN project. His mentorship extended beyond academic advice, as he kindly facilitated valuable connections with a range of radiologists and clinical technicians whose professional insights significantly enriched my understanding of medical imaging and contributed meaningfully to the direction and relevance of this work. His generosity, expertise, and commitment to supporting this project have been deeply appreciated.

I would also like to express my profound gratitude to Dr. Abubakr Siddig, my Machine Learning lecturer, for his invaluable evaluation, expert guidance, and continuous academic support. His insightful feedback, technical expertise, and encouragement greatly enhanced both the quality of this project and my understanding of Machine Learning concepts, particularly in the practical application of deep learning techniques. His mentorship and commitment to academic excellence have been truly inspiring throughout this process.

Finally, I would like to extend my heartfelt gratitude to my family and friends for their unwavering patience, encouragement, and continuous support. Their belief in my abilities has been a constant source of motivation and strength, enabling me to successfully complete this work.

Table of Contents

Acknowledgements ..... *iii*

List of Equations ..... *viii*

List of Figures ..... *viii*

List of Tables ..... *ix*

Abstract ..... *x*

Chapter 1. Introduction *1*

1.1 Medical Imaging and Challenges **1**

1.2 Tuberculosis and Pneumonia in Chest X-rays **2**

1.3 Global Impact **3**

1.4 Project Objectives **3**

1.5 Proposed Solution **4**

Chapter 2. Background *6*

2.1 Literature Review **6**

2.1.1 Current Status and Future Outlook of the European Radiology Workforce 6

2.1.2 Traditional v/s Deep Learning Methods in Diagnostic Radiology 6

2.1.3 Edge AI and Cross-Platform Integration in Clinical Settings 7

2.1.4 Supervised and Self-Supervised ML Approaches in Medical Imaging. 8

2.1.5 Vision Transformers (ViTs) and CNN 8

2.1.6 Challenges of Machine Learning in Health Care 9

2.2 Related Work **10**

2.2.1 Srivastava S, Ghosh E, Kumar A, Chahar P, Utkarsh A, Mishra R. Multi-class deep learning architecture for COVID-19, tuberculosis, and pneumonia classification using chest X-ray images. J Med Imaging Radiat Sci. 2025 Dec;56(6):102115. doi: 10.1016/j.jmir.2025.102115. Epub 2025 Oct 8. PMID: 41067063. 10

2.2.2 Georgios Kourounis, Ali Ahmed Elmahmudi, Brian Thomson, James Hunter, Hassan Ugail, Colin Wilson, Computer image analysis with artificial intelligence: a practical introduction to

convolutional neural networks for medical professionals, *Postgraduate Medical Journal*, Volume 99, Issue 1178, December 2023, Pages 1287–1294, https://doi.org/10.1093/postmj/qgad095 11

2.2.3 Medical Image Classification Using Machine Learning and Python by Srilakshmi Bathula on Medium Blog, 12th July 2025. https://medium.com/@lakshmi.sri.bath/medical-image-classification-using-machine-learning-and-python-8bd8d7bed7aa 11

2.2.4 CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning by Stanford ML group 12

Chapter 3. Methodology 13

3.1 Neurons and Convolutional Neural Network 13

3.2 Activation function in a neural network 13

3.3 Why CNN over other ML techniques? 14

3.4 Reason for TensorFlow and Keras over other ML frameworks 14

3.5 Why Flutter Cross-Platform Application? 15

3.6 Why Dart over other programming languages? 15

3.7 Common Radiological Features of Pneumonia 16

3.8 Common Radiological Features of Tuberculosis 16

3.9 Reason why the Batch Normalization layer is applied? 17

3.10 Relevance of Recall Score in Medical Evaluation 17

Chapter 4. System Design and Specifications 19

4.1 CXC Project Structure 19

4.2 Model Naming 20

4.3 Main Tech Stack 20

4.4 Backend Architecture 21

4.4.1 API Framework 22

4.4.2 Authentication and Authorization 23

4.4.3 System Variables 24

4.4.4 API Design Pattern 25

4.5 Frontend Architecture 27

4.5.1 Functional Requirements 27
4.5.2 Non-Functional Requirements 28
4.5.2 Frontend Design Pattern 29
4.5.3 Responsive & Adaptive 30
4.5.4 App Navigation 31
4.6 List of Endpoints 31
Chapter 5. Implementation 33
5.1 Image Dataset 33
5.2 Data Ingestion and Preprocessing 34
5.3 CXC 101 35
5.3.1 Model Architecture and Layer-wise Parameter Description 36
5.3.2 Model Fitting Observation. 39
5.4 CXC 102 39
5.4.1 Model Architecture and Layer-wise Parameter Description 40
5.4.2 Model Fitting Observation. 41
5.5 CXC 103 43
5.5.1 Model Architecture and Layer-wise Parameter Description 43
5.5.2 Model Fitting Observation. 45
5.6 CXC 201 45
5.6.1 Model Architecture and Layer-wise Parameter Description 46
5.6.2 Model Fitting Observation. 47
5.7 CXC 202 48
5.7.1 Model Architecture and Layer-wise Parameter Description 48
5.7.2 Model Fitting Observation. 49
5.8 CXC 203 50
5.8.1 Model Architecture and Layer-wise Parameter Description 50
5.8.2 Model Fitting Observation. 52
Chapter 6. Testing and Evaluation 54
6.1 Evaluation Criteria 54
6.2 CXC Model Evaluation Table 54

6.3 CXC 101 Model Evaluation **55**

6.4 CXC 102 Model Evaluation **56**

6.5 CXC 103 Model Evaluation **57**

6.6 CXC 201 Model Evaluation **59**

6.7 CXC 202 Model Evaluation **60**

6.8 CXC 203 Model Evaluation **61**

6.9 Application User Testing **63**

Chapter 7. Conclusions and Future Work ***64***

7.1 Conclusions **64**

7.2 Future Work and Enhancement **65**

References ***67***

Appendix I ***71***

Appendix II ***73***

List of Equations

Equation 1: formula for recall score. 17

List of Figures

Figure 1 : primary TB on the upper lobe [6] 2
Figure 2: Lobar Pneumonia consolidation [9] 3
Figure 3: Neuron representation 13
Figure 4 Project Structure connection view 19
Figure 5: FastAPI automated endpoint doc 22
Figure 6: Interaction UML diagram for login 23
Figure 7: API design structure 26
Figure 8: Initial dataset distribution 33
Figure 9 : Current dataset after modification 34
Figure 10: TensorFlow image_dataset_from_directory syntax 35
Figure 11: CXC 101 model accuracies 36
Figure 12: CXC 101 convolution layers 38
Figure 13: CXC 101 training loss graph 39
Figure 14: Image data augmentation 40
Figure 15: CXC 102 accuracy plot 42
Figure 16: CXC 102 loss graph 43
Figure 17: CXC 103 training accuracy v/s loss 45
Figure 18: CXC 202 fitting accuracy plot 49
Figure 19: CXC 203 model layers. 51
Figure 20: CXC 203 training loss over 10 epochs 52
Figure 21: CXC 203 training accuracy over 10 epochs 53
Figure 22: CXC 101 ROC curve 55
Figure 23: CXC 101 Confusion Matrix 56
Figure 24: CXC 102 classification report on test dataset 56
Figure 25: CXC 102 Confusion Matrix 57
Figure 26: CXC 103 confusion matrix 58

Figure 27: CXC 103 ROC Curve 59
Figure 28: CXC 201 Confusion Matrix 60
Figure 29: CXC 202 confusion matrix on test data 61
Figure 30: CXC 203 confusion matrix on test data 62

List of Tables

Table 1: Available Endpoints 32
Table 2 Model Comparison Table 54

Abstract

The Chest X-ray Classifier (CXC) project is an end-to-end deep learning system developed to automatically classify chest X-ray images into four categories: Normal, Pneumonia, Tuberculosis (TB), and Unknown. Built using Python, TensorFlow, and Keras, the project explores and compares both custom-designed Convolutional Neural Network (CNN) models and well-established transfer learning architectures such as ResNet50, VGG16, and InceptionV3. The objective was to evaluate how different network architectures, image resolutions, and training strategies affect classification performance. To improve the reliability of predictions, an additional Unknown class was introduced into the final dataset of 19,327 images, enabling the system to identify images that do not belong to the target disease categories and reducing incorrect classifications.

Among the custom CNN models developed in the CXC 100-series, CXC 103 achieved the best performance by incorporating Batch Normalization layers after convolutional operations. This model reached a test accuracy of 96.07% and a validation accuracy of 96.06%, demonstrating stable training and strong generalization. For the transfer learning models in the CXC 200-series, CXC 203, based on the ResNet50 architecture, delivered the highest overall performance with a test accuracy of 97.25% and a validation accuracy of 98.03%.

To make the system accessible and practical for real-world use, the trained models were integrated into a FastAPI backend secured using Firebase Authentication JSON Web Token (JWT) authentication. A responsive frontend cross-platform application was developed using the Flutter framework and the Dart programming language, allowing the application to run consistently across multiple platforms. The system supports single-image prediction, confidence score visualization, multi-model testing, and batch processing of chest X-ray images. Batch prediction results are automatically organized and exported as downloadable ZIP files, providing users with an efficient and user-friendly diagnostic support tool.

Chapter 1. Introduction

Modern technological advancements continue to progress rapidly, significantly enhancing the quality of human life [1]. In particular, the field of Machine Learning (ML) has evolved tremendously, leading to major improvements in medical image analysis and predictive capabilities. As ML models are trained on increasingly large and diverse datasets, they achieve higher levels of reliability and accuracy, giving them the potential to be suitable for real-world clinical applications.

Artificial Intelligence (AI), as a subset of machine learning, enables automated decision-making with minimal human intervention, improving efficiency and consistency in healthcare systems. These algorithms can perform tasks that were previously limited to human expertise, such as detecting subtle patterns and minute variations in medical images that may be overlooked by human vision [2]. This represents a major milestone in diagnostic science.

1.1 Medical Imaging and Challenges

Medical images include X-rays, CT scans, MRIs, ultrasound images, and other specialized imaging modalities used in clinical practice. Unlike conventional images captured using standard digital cameras, medical images are produced using dedicated medical instruments designed to capture internal structures of the human body. As a result, their quality, resolution, and characteristics differ significantly from natural images.

For instance, many medical images, such as X-rays, are single-channel (grayscale), meaning they represent variations in intensity using only black and white rather than multiple colour channels. These differences make medical image analysis more complex and require specialized machine learning techniques that can effectively handle variations in contrast, noise, and structure.

1.2 Tuberculosis and Pneumonia in Chest X-rays

TB is a serious bacterial or viral infection caused by Mycobacterium tuberculosis, which is contagious, meaning it can spread through the air. These mostly affect the lungs of humans. As many as 13 million people in the U.S have latent TB [3]. Similarly, Pneumonia is also a lung infection where tiny air sacs (alveoli) become inflamed and filled with liquid. Both are very fatal conditions.

Observing the presence of both conditions on a chest X-ray is purely based on black-and-white opacity pattern recognition [4]. TB is often seen as infiltrates, which are observable in the upper lobes of the human lungs. The size and shape of these infiltrates can vary widely depending on the stage and severity of the disease. So, an increase in lung opacity on chest X-rays might suggest active pulmonary TB [5].

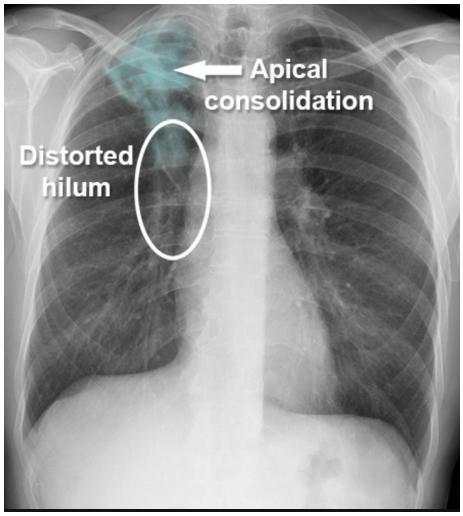


Figure 1 : primary TB on the upper lobe [6]

Whereas the presence of Pneumonia on X-ray appears with opacity in the lungs, often due to the accumulation of fluid or pus in the air sacs of the lungs. These areas can be lobar, diffuse, or patchy [7]. The lobar consolidations (cloudy lung regions) appear as dense, homogeneous opacities that obscure the margins of the lung lobes [8].

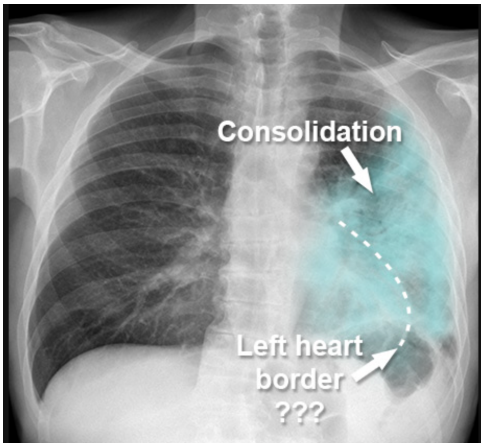


Figure 2: Lobar Pneumonia consolidation [9]

1.3 Global Impact

According to the World Health Organization (WHO), Pneumonia remains the leading infectious cause of death worldwide, responsible for approximately 2.5 million deaths annually [10]. Similarly, Tuberculosis remains one of the deadliest infectious diseases, affecting over 10 million people and causing more than 1.2 million deaths each year [11]. Most cases occur in developing regions and low-resource healthcare settings that lack sufficient diagnostic infrastructure.

The chest X-ray classifier project matters as it addresses a massive bottleneck in health care:

- The time it takes to get an X-ray read.
- The shortage of professional radiologists.

Recent studies suggest that the workforce gap between the demand for radiology services and the available radiologists is increasing [12].

1.4 Project Objectives

The main objective of this project is to design and develop an automated Chest X-ray classification system capable of identifying Normal, Pneumonia, and Tuberculosis

cases using deep learning techniques. The system aims to improve diagnostic efficiency by providing fast and accurate predictions from chest X-ray images.

A key objective of this project is also to evaluate the performance of custom CNN architectures, particularly in terms of how well they perform with fixed training epochs and reduced convolutional depth. This helps to understand the efficiency and limitations of lightweight CNN models when compared to more complex approaches.

In addition, the project focuses on implementing a scalable end-to-end system using FastAPI for model deployment and developing a user-friendly cross-platform Flutter application for clinical use. The system also supports batch processing of X-ray images to handle multiple cases efficiently, making it suitable for high-throughput medical environments. Overall, these objectives aim to improve accessibility to early-stage diagnostic support tools for healthcare professionals.

1.5 Proposed Solution

To address these challenges, this project focuses on the design, development, and implementation of the Chest X-ray Classifier (CXC) system-an end-to-end diagnostic application that automatically classifies chest X-rays into three categories: Normal, Pneumonia, and Tuberculosis. The system aims to support faster and more reliable diagnosis, helping healthcare professionals manage workload and reduce delays in identifying serious respiratory conditions.

At the core of the system is a supervised Convolutional Neural Network (CNN) designed for image classification. CNNs are deep learning models specifically suited for analysing visual data. Instead of relying on manually engineered features, the model learns directly from X-ray images. Early layers detect simple features such as edges and textures, while deeper layers learn higher-level medical patterns such as infiltrates, cavities, and consolidations. The system outputs probability scores across all three classes to produce a final prediction.

The project includes both custom-built CNN models (CXC 100-series) and transfer learning models based on architectures such as ResNet50 and VGG16 (CXC 200-series). These models were developed using Python, TensorFlow, and Keras to ensure scalability and performance.

To support real-world use, the models are deployed via a FastAPI backend and integrated into a cross-platform Flutter application. This allows users to upload single or multiple chest X-rays and receive rapid diagnostic predictions, enabling a more efficient clinical workflow.

Chapter 2. Background

2.1 Literature Review

2.1.1 Current Status and Future Outlook of the European Radiology Workforce

According to the recent European Union Radiation, Education, Staffing & Training (EU-REST) project [12], which was funded by the European Commission, the project shows that the gap between demand for radiology services and radiology supply is widening. The workforce is not growing enough, effectively creating a shortage by 2030. The Imaging demand is expected to increase by around 70% by 2030, while the radiologist headcount is growing only around 1% per year. That mismatch is the core reason for the widening gap. 45% of practising radiologists are already over age 51, implying a major retirement wave over the next decade that could worsen shortages if training pipelines do not replace them fast enough [13]. Radiologist density ranges from 51 to 270 radiologists per million population across EU countries. 16 of 27 EU countries are already below the EU average of 127 radiologists per million.

2.1.2 Traditional v/s Deep Learning Methods in Diagnostic Radiology

Traditional machine learning methods played an essential role in the early development of computer-aided diagnosis (CAD) systems in diagnostic radiology, especially for chest X-ray analysis. These methods relied heavily on manual feature extraction techniques like the Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG). Researchers used these techniques to find localized patterns, such as edges, textures, and shapes, in medical images before classifying them with algorithms like Support Vector Machines (SVM). In chest radiography, these techniques were often used to detect conditions like pneumonia and tuberculosis by manually isolating visual features from lung fields. [14].

The arrival of deep learning, particularly Convolutional Neural Networks (CNNs), directly addressed these limitations by enabling automated feature extraction directly from raw pixel data. Unlike shallow learning methods, CNNs automatically learn hierarchical image representations, beginning with low-level features like edges and progressing to high-level pathological patterns critical for clinical diagnosis. This capacity makes CNNs uniquely effective at identifying the faint, diffuse opacities characteristic of pulmonary infections that handcrafted descriptors often fail to capture. A key breakthrough in this field came from Pranav Rajpurkar[1] with the creation of CheXNet [15]. This 121-layer dense convolutional network achieved pneumonia detection results that matched, and in some cases surpassed, the average sensitivity of practicing radiologists. It was trained on a large, publicly available chest X-ray dataset.

2.1.3 Edge AI and Cross-Platform Integration in Clinical Settings

Recent healthcare research highlights the growing importance of Edge AI, where medical data is processed close to its source instead of relying only on centralized cloud systems. In hospitals, this approach reduces delays and supports faster decision-making for tasks such as patient monitoring, diagnostic support, and operational management. According to recent studies, Edge AI can improve healthcare responsiveness by enabling real-time analysis while also addressing concerns around privacy, network reliability, and computational efficiency. Cross-platform frameworks such as Flutter further support this shift by allowing one application to run across desktops, tablets, and mobile devices used by healthcare professionals [16].

An important consideration is whether to rely on on-device computation (Edge AI) or remote computation via solutions like FastAPI backends. Edge AI allows for quick inference while maintaining better data privacy, as no sensitive patient data leaves the device. This is ideal for compliance with health care regulations, however limits are imposed on the processing scale by the computational power of the device. On the

[1] https://arxiv.org/search/cs?searchtype=author&query=Rajpurkar,+P

other hand, server-based systems can process much larger quantities of information while simplifying the updates process to multiple hospitals [17]. As a result, many suggest a hybrid approach, where simple, time-sensitive diagnostic tools run on the device while more complex processing is handled through a protected cloud solution to take advantage of more available processing power.

2.1.4 Supervised and Self-Supervised ML Approaches in Medical Imaging.

Supervised uses a labelled dataset to train the ML model to understand the medical image, whereas unsupervised works with unlabelled data, it tries to discover patterns and structure without explicit guidance. Previously, supervised learning has shown significant accuracy. For instance, the 2022 article "Survey of Supervised Learning for Medical Image Processing" demonstrates this. However, Recent research and benchmarks like "Evaluating Self-Supervised Learning in Medical Imaging: A Benchmark for Robustness, Generalizability, and Multi-Domain Impact" [18] prefer self-supervised learning mainly due to limitations in annotated data. In addition, there is also proof that a combined supervised and unsupervised approach can improve the accuracy, one of which is "Improving image classification of gastrointestinal endoscopy using curriculum self-supervised learning" [19].

2.1.5 Vision Transformers (ViTs) and CNN

CNNs are highly effective for medical image segmentation because they use convolutional filters to capture local spatial patterns such as edges, textures, and shapes. This makes them well-suited for tasks like tumour detection, organ segmentation, and lesion classification, especially when trained on large labelled datasets. CNN-based and hybrid CNN models are therefore commonly used in clinical environments due to their strong performance, efficiency, and well-established training pipelines[20]. However, CNNs have a limited receptive field and may struggle to capture long-range dependencies in complex medical images.

In contrast, Vision Transformers (ViTs) process images as sequences of patches and use self-attention mechanisms to model global relationships across the entire image.

This allows ViTs to better capture long-range dependencies, which is particularly useful in radiology, where abnormalities may be spatially distributed or subtle. ViTs generally require large datasets for effective training, but can outperform CNNs in capturing global contextual information. Recently, hybrid models combining CNNs and ViTs have gained attention. These models leverage CNNs for local feature extraction and ViTs for global context understanding, leading to improved performance in medical imaging tasks such as segmentation, classification, and diagnosis. Overall, hybrid architectures often achieve better accuracy and robustness compared to using either CNNs or ViTs alone in radiology applications.[21].

2.1.6 Challenges of Machine Learning in Health Care

Machine learning has huge potential in healthcare, but in practice, there are several real-world challenges that make it difficult to apply safely and effectively. One of the biggest issues is the quality of medical data. Clinical data from electronic health records is often messy, incomplete, and inconsistently recorded. Doctors don't collect data in a uniform way-they only record what is needed for a patient at a specific time. So, datasets can have lots of missing values and noise. This makes it harder for models to learn accurate patterns.

Another challenge is unreliable or inconsistent labelling. In healthcare, diagnoses and medical codes are not always perfectly accurate, and the same condition can be recorded in different ways. On top of that, many diseases are not single conditions but have multiple subtypes, which adds further complexity for machine learning models [22].

There is also the problem of bias in the data. Some groups of patients are overrepresented while others are underrepresented, especially in rare diseases. This can lead to models that perform well in general but fail in high-risk or less common cases. A more fundamental challenge is that most machine learning models are very good at finding patterns, but healthcare often requires understanding cause and effect, not just correlations. For example, predicting a disease is not the same as

understanding whether a treatment will actually help a patient. Without this, models can make predictions that are not clinically useful.

Privacy and ethics are also major concerns. Medical data is highly sensitive, and strict regulations make it difficult to share large datasets. At the same time, models can unintentionally reflect or even amplify existing healthcare inequalities. Finally, there is the issue of trust and interpretability. Many advanced models, especially deep learning systems, are difficult for doctors to interpret. If clinicians cannot understand why a model made a prediction, they are less likely to rely on it in real decision-making [23].

Overall, while machine learning can greatly improve healthcare, these challenges around data quality, bias, causality, privacy, and trust need to be carefully addressed before widespread adoption

.

2.2 Related Work

2.2.1 Srivastava S, Ghosh E, Kumar A, Chahar P, Utkarsh A, Mishra R. Multi-class deep learning architecture for COVID-19, tuberculosis, and pneumonia classification using chest X-ray images. J Med Imaging Radiat Sci. 2025 Dec;56(6):102115. doi: 10.1016/j.jmir.2025.102115. Epub 2025 Oct 8. PMID: 41067063.

A similar study [24] was conducted for COVID-19, tuberculosis, and pneumonia classification on chest X-rays using CNN. They have used over 6,000 chest X-ray images across all classes and carried out transfer learning methods on a *RestNet-50* and *VGG-19* (19 weight layers) models. The study shows similar outcomes for the VGG-based model compared with other models. However, the *VGG-16* (16 weight layers) model used in this study has fewer parameters and a smaller depth, as per the Keras 3 Applications documentation. But even with lesser weights, the layer model, and with a smaller dataset, was able to achieve a close to 90% f1-score and recall matrix [25].

2.2.2 Georgios Kourounis, Ali Ahmed Elmahmudi, Brian Thomson, James Hunter, Hassan Ugail, Colin Wilson, Computer image analysis with artificial intelligence: a practical introduction to convolutional neural networks for medical professionals, *Postgraduate Medical Journal*, Volume 99, Issue 1178, December 2023, Pages 1287–1294, https://doi.org/10.1093/postmj/qgad095

A broader study of various medical diseases and conditions using ML CNN has been conducted. It assesses and classifies colorectal polyps and gastric epithelial tumours, as well as assists in the assessment of multiple malignancies [26]. Their study suggests that AI-based medical image analysis can reduce diagnostic turnaround time and improve efficiency in clinical assessments, highlighting the growing role of ML in healthcare diagnostics.

2.2.3 Medical Image Classification Using Machine Learning and Python by Srilakshmi Bathula on Medium Blog, 12th July 2025. https://medium.com/@lakshmi.sri.bath/medical-image-classification-using-machine-learning-and-python-8bd8d7bed7aa

A study by Srilakshmi Bathula[2] used machine learning and deep learning models such as ResNet50 and VGG16 for medical image classification and achieved an accuracy of around 91-95%. The study also used image preprocessing and transfer learning techniques to improve results. In comparison, this project achieved better performance, with the custom-built model reaching around 96% evaluation accuracy, while the ResNet50 model achieved around 97% accuracy using only 10 epochs of training. This shows that the proposed approach provides higher accuracy and better performance with fewer training iterations compared to the related study.

[2] https://medium.com/@lakshmi.sri.bath

2.2.4 CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning by Stanford ML group

CheXNet, developed by the Stanford ML Group, is a major benchmark in automated chest X-ray analysis. It is built using a deep 121-layer DenseNet architecture and trained on the large ChestX-ray14 dataset, which contains over 100,000 images covering 14 thoracic conditions. One of its key achievements was strong performance in pneumonia detection, where it matched and, in some cases, exceeded the performance of practicing radiologists. The study also introduced Class Activation Maps (CAMs), which provide visual explanations by highlighting the regions of the X-ray that influenced the model's prediction [27].

When compared with the CXC project, there are clear differences in dataset scale and model complexity. CheXNet is trained on a much larger dataset and uses a significantly deeper and more complex architecture. In contrast, the CXC models are trained on a smaller dataset and use simpler architectures, including custom CNNs and transfer learning models such as ResNet50 and VGG16, adapted for a more focused classification task involving Normal, Pneumonia, and Tuberculosis.

Despite these differences, both approaches aim to improve chest X-ray diagnosis using deep learning. CheXNet demonstrates performance at a large-scale research level, while the CXC models show how lighter architectures can still achieve strong results on smaller datasets, particularly when applied to specific medical classification tasks.

Chapter 3. Methodology

3.1 Neurons and Convolutional Neural Network

Neurons are the most basic unit in a neural network. They consist of a combination of linear and non-linear functions called activation functions. A neuron receives input data, processes it by assigning weights and biases, and produces an output. It is inspired by how biological neurons in the human brain work, where signals are received, processed, and transmitted to other neurons [28].

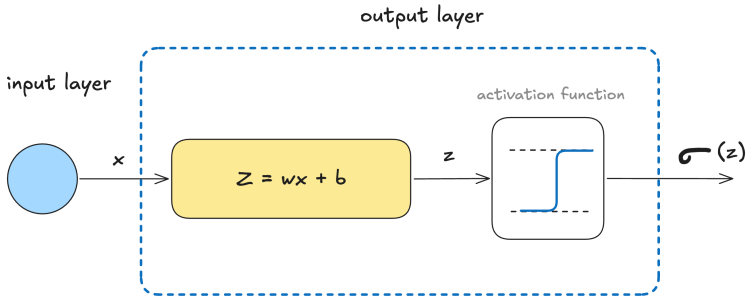


Figure 3: Neuron representation

A Convolutional Neural Network (CNN) is a type of neural network mainly used for image processing and computer vision tasks. CNNs are designed to automatically detect important features from images, such as edges, shapes, and textures, through convolution operations. Unlike traditional neural networks, CNNs use layers such as convolution, pooling, and fully connected layers to extract patterns efficiently. They are widely used in applications like image recognition, facial recognition, object detection, and medical image analysis [29].

3.2 Activation function in a neural network

In the context of machine learning, an activation function acts as a crucial decision maker within a neural network, determining whether a specific neuron should "fire" or

pass its information forward. Mainly, its primary purpose is to introduce non-linearity into the network.

Without this non-linear property, no matter how many hundreds of layers are stacked together, the entire neural network would collapse mathematically into a single, basic linear regression model [30]. It would be fundamentally limited to performing simple operations-essentially just drawing straight lines-rendering it incapable of capturing the complex, intricate patterns found in real-world data like medical images, text, or audio. By bending and transforming the data, activation functions allow deep learning models to learn and solve highly sophisticated, non-linear problems.

3.3 Why CNN over other ML techniques?

Convolutional Neural Network, also known as CNN, is a mixed technique of Machine Learning and Deep Learning algorithms that recognises patterns such as edges, shapes, and objects, in images and videos. CNN preserves the spatial relationships in the image using convolutional operations, i.e., they understand where abnormalities occur in the lung. This is very critical for detecting the disease patterns in tuberculosis and pneumonia because these patterns are location sensitive.

3.4 Reason for TensorFlow and Keras over other ML frameworks

TensorFlow and Keras are considered the industry standard for machine learning due to their robust, end-to-end ecosystem that prioritizes production-grade deployment and enterprise scalability. Keras provides a high-level, "human-centric" API that accelerates prototyping through modularity and simplicity. It provides various APIs for preprocessing and data loading. Since medical image datasets can be large and computationally intensive, TensorFlow's optimized processing capabilities make it highly suitable for healthcare AI applications.

What makes it different from other frameworks, such as Pytorch, is that it needs more configurations as compared to TensorFlow. Plus, TensorFlow provides various tools

like TensorFlow Board for great visualization, and TensorFlow Extended (TFX) for managing full ML pipelines.

3.4 Cross Categorical

3.5 Why Flutter Cross-Platform Application?

Flutter [3]enables cross-platform development framework from a single code base, and it is by Google. By cross-platform, it means that it can work on various platforms like Web, Android, IOS, Windows, Mac OS, and Linux. Flutter applications are very scalable and reliable in production. The framework uses the Dart programming language. It is considered scalable and reliable for production systems. In addition, Flutter has an active developer community, which provides strong support, regular improvements, and shared resources. The framework also receives security patches and updates, making it suitable for healthcare applications that require secure handling of sensitive patient data.

3.6 Why Dart over other programming languages?

Dart was selected as the programming language for the frontend because it is specifically designed for building client-side applications that users interact with directly. It supports the development of mobile, desktop, web, and command-line applications while focusing on both developer productivity and user experience. Since the project required a responsive and cross-platform application, Dart provided a suitable solution by allowing development through a single codebase while maintaining consistent functionality across different platforms.

One of the main advantages of Dart is its clean and easy-to-understand syntax, which makes the development process faster and more manageable. This improves developer productivity by making the code easier to write, read, and maintain throughout the project lifecycle. Dart also integrates seamlessly with Flutter, enabling efficient

[3] https://flutter.dev/

frontend development and simplifying the creation of modern and interactive user interfaces.

Another important reason for choosing Dart is its strong performance and reliability. Dart applications are compiled into native machine code, allowing them to run efficiently with smooth interactions and responsive animations. In addition, Dart offers type safety and null safety, reducing common runtime errors and improving application stability. These features contribute to a more reliable, maintainable, and high-performing application.

3.7 Common Radiological Features of Pneumonia

For Pneumonia, one common finding is lobar consolidation, which appears as a dense, uniform white area affecting a lung lobe and represents inflammation filling the alveoli [31]. Another pattern is bronchopneumonia, which presents as patchy areas of infection distributed around the bronchi and bronchioles and may present in multiple regions of the lungs [32]. The Lobar consolidation or patchy opacities are generally distributed in the lower lobe areas [8]. However, these radiological features may not be clear in a dehydrated patient’s lung X-ray image [7].

3.8 Common Radiological Features of Tuberculosis

For Tuberculosis, a range of radiological features depend on whether the infection is primary or post-primary. The primary TB is the first time someone gets infected, and it mostly affects the middle or lower parts of the lungs. It shows up as patchy fluid patches (consolidation) and swollen lymph nodes in the chest. As it heals, it can leave behind small, hard scars made of calcium. For post-primary TB, the most common sign is the formation of hollow holes in the lung tissue (cavities) caused by tissue damage, alongside scarring, airway damage, and fluid buildup. It can also spread into the airways, creating a distinct "tree-in-bud" pattern on CT scans [33] [34].

3.9 Reason why the Batch Normalization layer is applied?

As the deep neural network has a training problem, as the data flows layer by layer, the distribution of activation shifts. What has started as normalized input become widely skewed over the layers. Some neurons are saturating, and others might barely be activating. Therefore, gradients can't flow properly, and training slows to a crawl, or explodes entirely. Batch normalization fixes this by doing something elegant, after each layer, before the activation function, it normalises the output across the current batch. Subtract the mean divided by the standard deviation, now the activations are centred around zero with unit variance. Then it adds two learnable parameters, scale and shift. So the network can undo the normalization if that's actually what it needs. The result is that you could train ten times deeper networks and with much higher learning rates. Thus, it makes the training more stable. Most of the modern ML architecture uses batch normalization.

3.10 Relevance of Recall Score in Medical Evaluation

Recall Score, also known as "Sensitivity", shows how effectively a model can detect cases that truly belong to the positive class. It indicates the model's ability to find and correctly identify all relevant positive instances, helping to measure how many actual positive cases are successfully captured rather than being missed. In medical applications, this means evaluating how well the model identifies patients who genuinely have a particular disease or condition.

$$Recall = \frac{TP}{TP + FN}$$

Equation 1: formula for recall score.

Where TP (True Positive) are the positive cases correctly identified. FN (False Negative) are the positive cases incorrectly classified as negative. The recall score ranges from 0 to 1 (or 0% to 100%). A higher recall means the model misses fewer actual positive cases.

It is very important in medical diagnosis because failing to detect a disease can have serious consequences. For example, when diagnosing for conditions such as TB or pneumonia, a false negative result may cause a patient with the disease to be classified as healthy. This can delay treatment and worsen health outcomes. Therefore, medical ML models often prioritize high recall to ensure that as many patients with the condition as possible are identified.

Chapter 4. System Design and Specifications

4.1 CXC Project Structure

The project is organized into several repositories: datasets, backend, models, and frontend, with each serving a specific purpose in the system. The datasets directory stores all image data used for training, validation, and testing the models. These datasets provide the foundation for training the system to accurately classify medical images.

The backend directory contains the code for the system's API, which manages communication between the frontend and the machine learning models. It also includes a Notebook directory that provides documentation and development records for all CXC models, making it easier to track experiments and model performance. The models directory stores all trained CXC models in .keras format. These models are loaded and used by the backend to make predictions, and newly trained models can also be saved back into this directory.

Finally, the frontend directory contains the Flutter application code for the CXC system. This is the user-facing part of the application, allowing users to interact with the platform. The frontend communicates with the backend to send requests and receive model predictions, while the backend relies on the datasets and stored models to process and generate results.

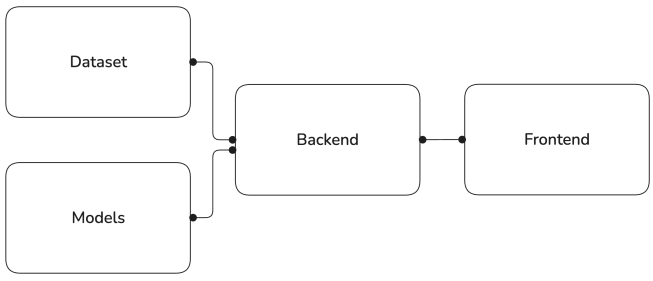


Figure 4 Project Structure connection view

4.2 Model Naming

Since the goal is to classify the chest X-rays, the CNN models are named as CXC (Chest X-ray Classification). The initial model names were CXC 101, followed by CXC 102, and more. Likewise, there are various iterations of the model. The custom layered models are built, and names such as CXC1xx series and transfer learning models are CXC2xx series. Each resolves the issues of its predecessor mode. The models are stored in the “models” directory, and the model paths are fetched from the system environments for training and for API purposes.

4.3 Main Tech Stack

The project uses a range of technologies to support machine learning, application development, and user management. For the backend, Python is used as the primary programming language because of its flexibility and strong ecosystem for AI and web development. The machine learning models and data processing pipeline were developed using TensorFlow and Keras, which were used to train, evaluate, and fine-tune the models.

During the experimentation phase, Jupyter Notebooks were used to test different approaches, visualize results, and document the development process. This made it easier to track model performance and compare improvements throughout the project. To integrate the machine learning models into the application, FastAPI was used to build the system API. This allows the trained models to be accessed efficiently by the application and enables smooth communication between the frontend and backend.

For the frontend, Dart was chosen as the programming language, while Flutter was used as the framework to develop a cross-platform application. This approach made it possible to build a single application that can run across multiple platforms while maintaining a consistent user experience. For user authentication and account management, Google Firebase was integrated into the system, providing a secure and reliable way to handle user sign-in and authentication.

In addition to the core development technologies, Git was used for version control throughout the project lifecycle. Git made it possible to track code changes, manage different development branches, and maintain a clear history of project updates. This was particularly useful for monitoring progress, reverting changes when necessary, and ensuring stable development practices.

To store and manage the project repository, GitHub was used as the remote hosting platform. GitHub provided a centralized location to securely store the project source code, manage version history, and facilitate collaboration and backup. It also ensured that development progress was consistently maintained and accessible throughout the project.

4.4 Backend Architecture

The backend system was designed using a structured and modular architecture to support the core functionality of the application while ensuring maintainability, scalability, and efficient system performance. It is responsible for handling requests from the frontend, processing application logic, managing communication between different system components, and returning responses to users. The architecture is organised into separate modules to maintain a clear separation of responsibilities, allowing different functionalities to operate independently while working together as part of a unified system. This modular approach improves code organisation, simplifies maintenance, and makes future system enhancements easier to implement. Additionally, the backend incorporates a logging mechanism to monitor system activities, application behaviour, and potential errors during execution. Logs are systematically stored to support debugging, performance monitoring, and issue tracking, helping improve system reliability and making maintenance more efficient.
In addition to the core backend components, the system also includes supporting resources for experimentation, documentation, and model development. During the development process, notebooks were used to document model creation, experimentation, training, and evaluation activities. These notebooks provided a structured environment for testing different approaches, analysing results, and

recording observations throughout the model development lifecycle. Together, the backend architecture, logging system, and supporting documentation contribute to a well-organised system that supports both application functionality and continuous improvement.

4.4.1 API Framework

To make the model available for testing and practical use, a backend API was developed in Python using the FastAPI framework. This API acts as the connection between the machine learning model and the application, allowing users to send requests and receive predictions efficiently.

One of the key advantages of using FastAPI is its built-in support for automatic API documentation and testing. Each REST endpoint is automatically documented and comes with an interactive interface, making it easy to test API functionality directly in the browser. This removes the need for additional API testing tools, such as Postman, during development and testing.

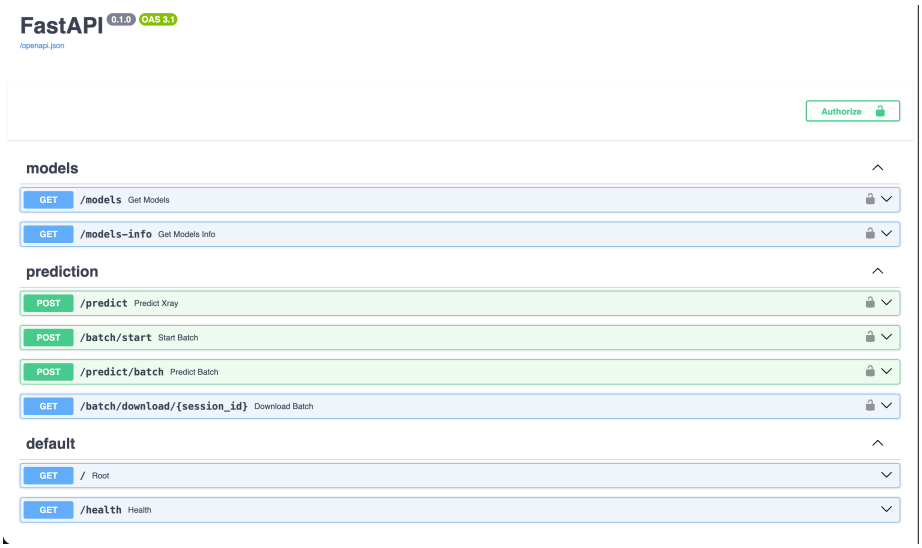


Figure 5: FastAPI automated endpoint doc

4.4.2 Authentication and Authorization

To protect system resources and prevent unauthorized access to the backend API, all endpoints are secured using Firebase-issued JSON Web Tokens (JWTs) through the HTTP Bearer authentication scheme. Once a user successfully logs in through Firebase on the frontend, a unique Firebase ID token is generated for that session. This token must be included in the authorization header of every subsequent API request in the format "Authorization: Bearer <token>".

When a request reaches the backend, a dedicated user verification middleware, implemented as the "*get_current_user*" dependency within the API middleware package, automatically intercepts the request and validates the user's identity. The middleware verifies the authenticity of the token by cryptographically checking it against Google's public key certificates linked to the configured "FIREBASE_PROJECT_ID" in the environment variable.

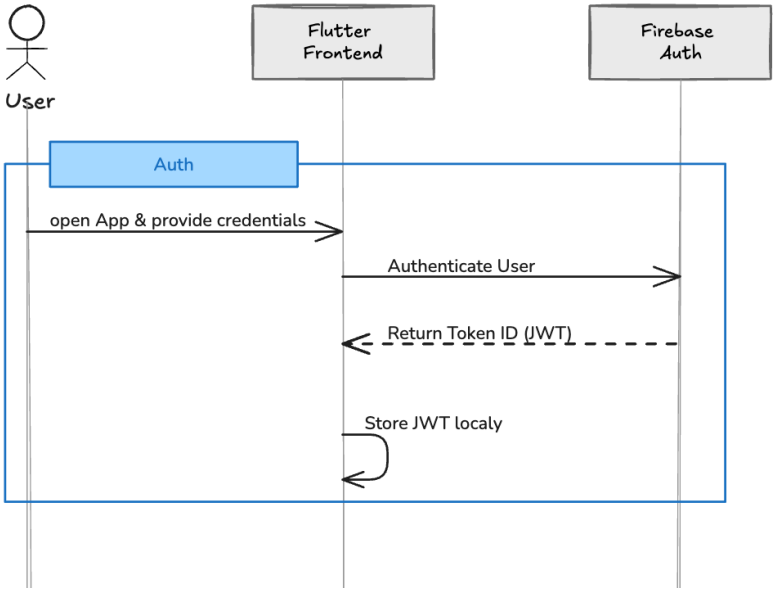


Figure 6: Interaction UML diagram for login

To ensure consistent security across the application, this middleware is applied directly at the FastAPI router level. As a result, all child endpoints are automatically

protected, meaning user credentials are verified before any request is processed. If a token is missing, invalid, or unauthorized, the request is immediately rejected with an HTTP 401 unauthorized response.

4.4.3 System Variables

To improve security and maintainability, all system variables and sensitive configuration values are stored securely in a ". env" file rather than being hardcoded directly into the source code. These variables are dynamically loaded at runtime through a dedicated variables file located within the constants directory. This approach allows the application to access important configuration settings, such as API keys, database connection strings, authentication secrets, and Firebase project credentials, without exposing them in the codebase.

Using environment variables provides an additional layer of security by separating sensitive information from the application logic. As a result, confidential credentials are not directly visible to developers or users who may access the source code repository. Even if the source code is accidentally shared or exposed, critical secrets remain protected because they are stored externally and are not included in the main application files.

This design also improves flexibility across different environments, such as development, testing, and production. For example, separate .env files can be configured with different values depending on the deployment environment, allowing the system to adapt without requiring changes to the application code. This reduces the risk of configuration errors and makes the deployment process more secure and manageable.

In addition, the .env file is typically excluded from version control systems using files such as ".gitignore", ensuring that sensitive credentials are not accidentally uploaded to public repositories. By following this approach, the system strengthens security,

simplifies configuration management, and reduces the likelihood of credential exposure.

4.4.4 API Design Pattern

The backend follows a clean and modular API design pattern to ensure better maintainability, scalability, and separation of concerns. The overall structure is organized in a layered approach, where each layer has a specific responsibility within the system.

At the top level, all API endpoints are defined in the routes directory. This layer acts as the entry point for all incoming requests. The endpoints are grouped based on functionality to keep the codebase organised and easy to navigate. For example, all prediction-related endpoints are defined in PredictionRoute.py, while model-related endpoints are managed in ModelRoute.py. This separation ensures that each file has a clear and focused responsibility.

The routes layer does not contain business logic. Instead, it delegates processing to the services layer, which contains the core application logic. The services are responsible for handling operations such as processing requests, running predictions, and managing model workflows. This design helps keep the API layer lightweight and improves code reusability and testability.

The services layer, in turn, makes use of shared helper methods from the utils package. These utility functions handle common operations such as data preprocessing, validation, formatting, and other reusable logic components. By centralising these functions, the system avoids code duplication and ensures consistency across different services.

Configuration values, constants, and system-wide parameters are managed separately in the constants package. This ensures that important values such as model thresholds, file paths, and environment-specific settings are maintained in a single location, making updates easier and reducing the risk of inconsistencies across the application.

For machine learning model management, the system includes a dedicated register package, which contains a model submodule. This component is responsible for storing model metadata, configurations, and specifications. It acts as a central registry for all available models, making it easier to manage different versions and switch between models when required.

Overall, this layered architecture comprising routes, services, utilities, constants, and model registration ensures a clear separation of responsibilities. It improves code readability, makes the system easier to maintain, and supports future scalability as new features or models are added.

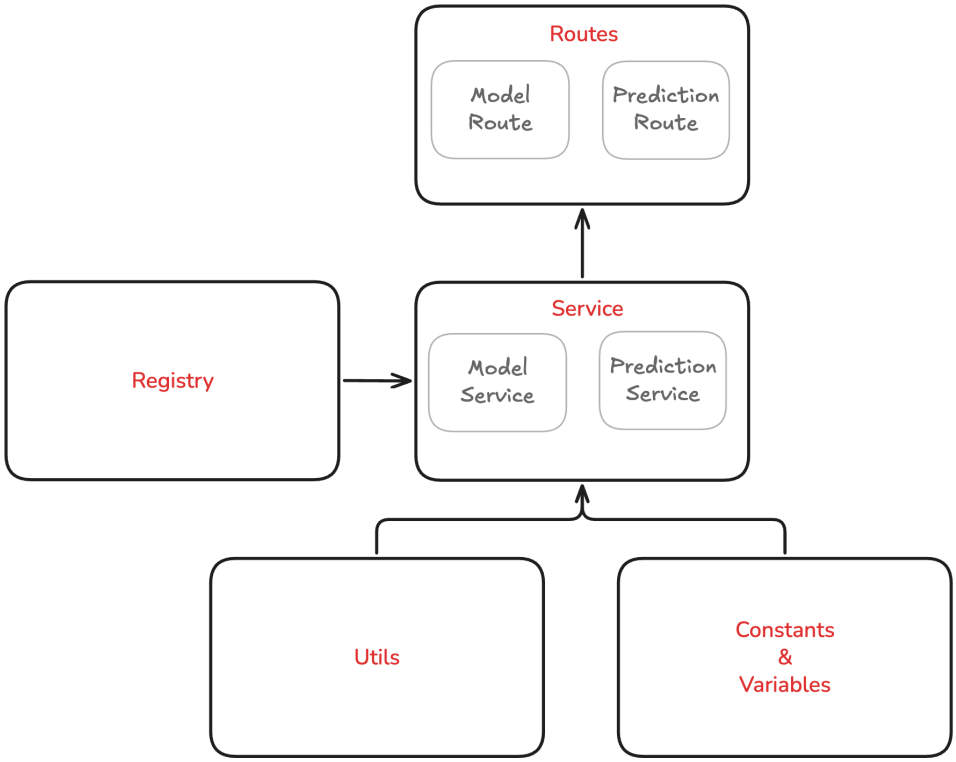


Figure 7: API design structure

4.5 Frontend Architecture

4.5.1 Functional Requirements

The frontend system was designed to provide users with an intuitive and efficient interface for interacting with the chest X-ray classification platform. The following functional requirements define the key features and capabilities expected from the frontend application:

- **User Authentication**
  The system should allow users to create an account and securely log into the application. Users should be able to sign up using valid credentials and access the system through a secure authentication process.
- **Account Management**
  The user should be able to manage their account, including the option to permanently delete their account from the system when required.
- **Single Chest X-ray Prediction**
  The system should allow users to upload a single chest X-ray image and perform disease classification using a CNN-based machine learning model. After processing, the prediction result should be displayed clearly to the user.
- **Prediction Result Visualization**
  The system should display prediction outcomes using an interactive graphical representation. Users should be able to view the confidence percentage for each predicted class as well as the final selected class label generated by the model.
- **Multi-Model Testing**
  The system should enable users to evaluate the same chest X-ray image using multiple available machine learning models. This functionality allows users to compare prediction outcomes and observe differences between model performances.
- **Batch Chest X-ray Prediction**

The system should support predictions for multiple chest X-ray images simultaneously. After processing, the system should automatically categorize or segregate the images based on prediction outcomes and provide the results in a downloadable ZIP file for user convenience.

4.5.2 Non-Functional Requirements

The CXC Flutter application was designed with several non-functional requirements to ensure a smooth, efficient, and user-friendly experience. These requirements focus on the overall quality and performance of the system rather than its core functionality.

- **Prediction Response Time**

  The system was designed to provide prediction results within a short period after image submission. Optimization techniques were applied to minimize delays and ensure that users receive classification results quickly for both single-image and batch prediction tasks.

- **Model Caching**

  To improve prediction speed, machine learning models are loaded and cached in memory when the backend prediction service starts. This eliminates the need to reload models for every prediction request, significantly reducing response time and improving overall system performance.

- **Adaptive and Responsive User Interface**

  The frontend was developed using Flutter to provide a responsive and adaptive user interface. The application automatically adjusts its layout and components according to different screen sizes, ensuring a consistent user experience across mobile phones, tablets, and desktop devices.

- **Multi-Platform Support**

The system was designed as a cross-platform application, allowing it to run on Android, iOS, Windows, macOS, and web platforms from a single codebase. This approach improves accessibility while reducing development and maintenance effort.

4.5.2 Frontend Design Pattern

The project follows a Feature-First structure, meaning the code is organised around different app features instead of being separated by technical roles like UI, services, or controllers. In simple terms, everything related to a specific feature is kept together in one place, making the project much easier to navigate and manage.

This approach works especially well in Flutter because Flutter is built around reusable widgets. Since the app is designed in a modular way, each feature can have its own widgets, logic, and supporting files grouped together. This makes development more organised, as changes to one feature are less likely to affect other parts of the application.

Using this design pattern also makes the app easier to scale. As new features are added, developers can create them as separate modules without cluttering the overall project structure. It also improves maintainability and debugging, because developers can quickly find all the files related to a feature without searching across multiple folders. Overall, this modular and widget-based structure helps keep the Flutter application clean, flexible, and easier to grow over time.

Inside each module within “lib/features/”, the codebase follows a clear separation of responsibilities across four layers, similar to a clean Model-View-Controller (MVC) or Model-View-ViewModel (MVVM) architecture:

- **Model**: Contains simple Dart data classes (e.g., *ModelInfo*) that define data structures and handle JSON conversion.

- **View**: Manages the Flutter UI, split into pages (full screens) and reusable widgets (smaller UI elements such as cards, chips, or grids).
- **Controller**: Handles component state, manages user interactions, and coordinates logic by calling the required services (e.g., *DocsController*).
- **Service**: Communicates with external systems, such as REST APIs, or accesses device features like the camera, image picker, and secure token storage.

This layered approach keeps the code organised, easier to maintain, and simpler to scale as features grow.

4.5.3 Responsive & Adaptive

The project uses a responsive layout system with clearly defined breakpoints. These are managed centrally through *ResponsiveHelper* and *Breakpoints*, which help the app identify the type of device and adjust the interface accordingly.

A key part of this responsiveness is Flutter's **LayoutBuilder**[4]. LayoutBuilder allows widgets to respond to the amount of space available at runtime rather than relying on fixed screen sizes. It works by providing layout constraints, such as width and height, so the UI can rebuild itself based on the current screen dimensions. For example, in components like *ModelInfoImageSection*, LayoutBuilder is used to switch between different layouts depending on available space, displaying content in rows or wrapped layouts on tablets and desktops, while automatically falling back to a single-column list on smaller mobile screens. This helps ensure the interface remains readable and user-friendly across devices.

The app also maintains a consistent look and feel by dynamically adjusting spacing and text sizes. Helpers such as *ResponsiveHelper.getResponsivePadding()* and

[4] https://api.flutter.dev/flutter/widgets/LayoutBuilder-class.html

getResponsiveFontSize() are used to scale margins, padding, and typography based on screen size, ensuring the UI feels balanced and accessible regardless of device.

4.5.4 App Navigation

The app uses a Navigation Shell (*MainShell*) to create a smooth, native-like navigation experience across the main sections. Instead of rebuilding pages every time the user switches tabs, it uses an *IndexedStack* to keep all main screens alive in the background. This ensures that when a user navigates between sections, the UI remains responsive and the state is preserved rather than being recreated.

As a result, each page retains its current state. For example, scroll positions, partially filled forms, and previously loaded data remain intact when the user returns to that page. This makes the experience feel seamless and efficient.

In addition, all application routes are centrally managed in a dedicated routes file, which keeps navigation logic clean, organised, and easy to maintain. This separation of routing from UI components improves scalability and makes it simpler to add or modify screens without affecting the overall structure. Overall, this approach ensures both performance and maintainability while delivering a smooth user experience.

4.6 List of Endpoints

These are the relevant API backend endpoints and their purposes.

Request Type	Endpoint Name	Purpose
GET	/health	Check if the backend is alive
GET	/models	For returning list of available models. It only list the model alias.
GET	/get_model_info	For returning list of models and their metadata information's from model registry.
POST	/predict	It provides prediction class and confidences for given file and specified model.
POST	/batch/start	It returns the batch prediction session id, for temporary storage.
POST	/predict/batch	It returns multiple images using a single batch inference request, for given session id, list of files and specified model.
GET	/batch/download	Create a ZIP file of classified images and return it for download. Temporary files are removed after the download completes.

Table 1: Available Endpoints

Chapter 5. Implementation

5.1 Image Dataset

Firstly, the image data is sourced from various authors on Kaggle[5], which is a publicly available dataset. For this study, the majority of the image dataset is used from the author *Rifatul Islam Majumder*[6], but the basic image (Figure 8) dataset undergoes some modifications, like the number of images for training is increased. The dataset is grouped into test, train, and val (validation). The training data contains a total of 19,327 images, of which 6,038 are normal X-ray images, 6,165 are Pneumonia images, 5,927 are TB images, and 1155 are random unknown images. During each training epoch, the validation accuracy and loss on the validation (val) dataset are also evaluated. The val dataset contains a total of 761 images, out of which 274 are normal chest X-ray images, 213 are images of Pneumonia, 207 are images of TB, and 57 are random unknown images. After which, the model is tested against the test dataset, which is biased and not exposed while training, and contains a total of 1527 images, of which 548 are normal, 427 are Pneumonia images, 417 are TB, and 135 are unknown images. The image size varies in size, but most of the images are 224 × 244 pixels. Figure 9 shows the image data distribution table for the respective classes.

Dataset	Normal	Pneumonia	Tuberculosis	Unknown	Total
Train	4667	3633	3573	1155	13028
Test	548	427	417	135	1527
Val	274	213	207	67	761

Figure 8: Initial dataset distribution

[5] https://www.kaggle.com/

[6] https://www.kaggle.com/rifatulmajumder23/datasets

Dataset	Normal	Pneumonia	Tuberculosis	Unknown	Total
Train	6083	6165	5924	1155	19327
Test	548	427	417	135	1527
Val	274	213	207	67	761

Figure 9 : Current dataset after modification

The unknown class was added on the assumption that the end user of the ML model application might attempt to test an image that is not an image of X-ray or could possibly test any other X-ray that is not a chest X-ray. So, to avoid possible incorrect prediction or distribution of confidence to main classes, it is better to keep an extra unknow classes. So, when predicting, all images other than Chest X-ray images will fall into the unknown category. Hence, it increases the reliability of the application in the real world.

5.2 Data Ingestion and Preprocessing

To load the image data efficiently, the TensorFlow *image_dataset_from_directory()* utility API was used. This function automatically reads images directly from the directory structure, assigns labels based on folder names, and converts them into a TensorFlow dataset format suitable for deep learning workflows. This approach simplifies data loading and reduces the need for manual preprocessing.

During data ingestion, all images were loaded in Red Green Blue (RGB) color mode (using the *color_mode* parameter*)*, ensuring that each image contained three color channels. On loading, the images were resized to a predefined dimension using the *image_size* parameter. This ensured consistency across all images before they were passed into the model.

The dataset was also processed in batches using the *batch_size* parameter. Batch processing improves computational efficiency by allowing multiple images to be

processed simultaneously during training and evaluation, reducing memory overhead and improving model optimization.

Furthermore, the *label_mode='categorical'* configuration was applied to generate labels in one-hot encoded format, which is suitable for multi-class classification problems. Since the model predicts across four different classes, categorical labels enable the model to calculate classification probabilities effectively using categorical loss functions.

An example of the test dataset loading implementation is shown in Figure 10 below:

```
test_ds = tf.keras.utils.image_dataset_from_directory(
    test_data_dir,
    color_mode='rgb',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode='categorical'
)
```

Figure 10: TensorFlow image_dataset_from_directory syntax

In this implementation, the test dataset was loaded from the test_data_dir directory and automatically pre-processed into batches of resized RGB images with categorical labels. Similar steps were applied to the training and validation datasets to maintain consistency throughout the model training pipeline. By using TensorFlow's built-in dataset utility, the data ingestion process became more efficient, scalable, and easier to integrate with the training workflow.

5.3 CXC 101

This is the initial experimental CNN model developed for a four-class image classification task using a total of 13,028 RGB images with an input resolution of 224 × 224 × 3. The model achieved approximately 83% training accuracy and 88.04%

validation accuracy when evaluated on a testing dataset containing 761 images. The model gradually increased the accuracy over each epoch, as shown in Figure 11.

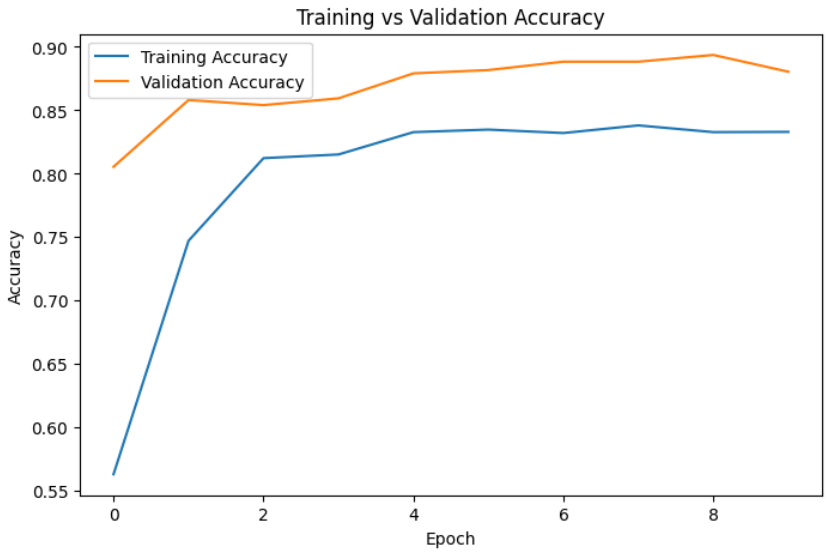


Figure 11: CXC 101 model accuracies

5.3.1 Model Architecture and Layer-wise Parameter Description

The architecture begins with a Batch Normalization layer, which standardizes the input distribution and helps stabilize the training process. This layer contains 12 parameters, corresponding to the scaling and shifting variables applied across the three image channels.

The feature extraction stage consists of three convolutional blocks, where each block includes a Conv2D layer followed by a MaxPooling2D layer. The convolutional layers are responsible for learning spatial features such as edges, textures, shapes, and higher-level visual patterns from the input images.

The first convolutional layer uses 32 filters with a kernel size of 3 × 3 and applies the *sigmoid* activation function. This layer produces an output feature map of size 222 222 × 32 and contains 896 trainable parameters. The number of parameters is calculated based on the convolutional kernel weights and bias terms:

$$(3 \times 3 \times 3 \times 32)\ 32 = \mathbf{896}$$

A *MaxPooling2D* layer with a pooling size of (2,2) is then applied to reduce the spatial dimensions from 222 × 222 to 111 × 111, which decreases computational complexity while preserving important features.

The second convolutional block consists of a Conv2D layer with 64 filters and the *ReLU* activation function. *ReLU* is computationally efficient and helps improve deep feature extraction. This layer outputs feature maps of size 109 × 109 × 64 and contain 18,496 parameters:

$$(3 \times 3 \times 32 \times 64) + 64 = \mathbf{18{,}496}$$

Another MaxPooling2D (2,2) operation downsamples the feature maps to 54 × 54 × 64. The third convolutional block applies 128 Conv2D filters with the *ReLU* activation function, generating feature maps of size 52 × 52 × 128. This layer contains 73,856 trainable parameters:

$$(3 \times 3 \times 64 \times 128) + 128 = \mathbf{73{,}856}$$

The subsequent pooling layer further reduces the dimensionality to 26 × 26 × 128, allowing the network to retain the most discriminative features while reducing memory usage and overfitting risk.

After the convolutional feature extraction process, the multidimensional feature maps are transformed into a one-dimensional vector using a Flatten layer. This operation converts the output into a vector of size 86,528, enabling it to be processed by fully connected layers.

The flattened vector is then passed into a Dense layer with 128 neurons, which serves as the primary classification learning layer. This fully connected layer contains

11,075,712 parameters, representing most the model's trainable weights due to the large flattened input dimension:

$$(86{,}528 \times 128) + 128 = \mathbf{11{,}075{,}712}$$

To reduce overfitting and improve model generalization, a Dropout layer with a dropout rate of 0.5 is applied. This means that 50% of the neurons are randomly deactivated during training, preventing the network from becoming overly dependent on specific features.

Finally, the output layer consists of a Dense layer with 4 neurons and a Softmax activation function, enabling multiclass probability prediction across the four target classes. The output layer contains 516 parameters:

$$(128 \times 4) + 4 = \mathbf{516}$$

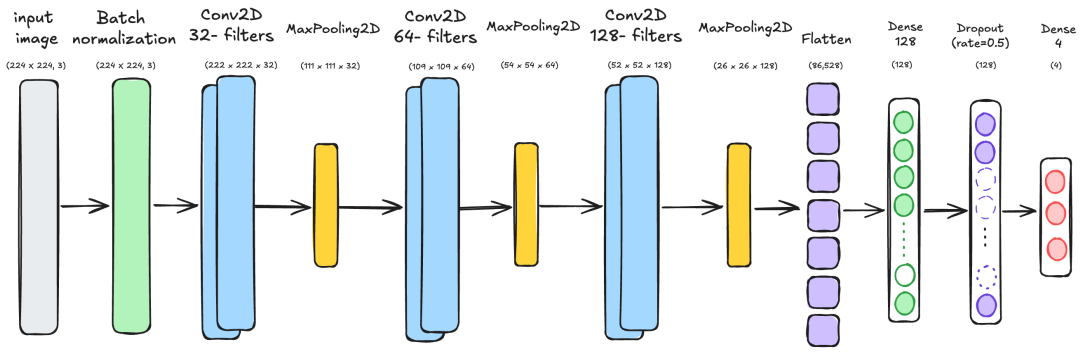


Figure 12: CXC 101 convolution layers

Overall, the CNN architecture contains a total of 33,508,454 parameters, of which 11,169,482 are trainable parameters, and 6 are non-trainable parameters. The remaining parameters are associated with the optimizer states maintained during training. The combination of convolutional feature extraction, pooling-based dimensionality reduction, dropout regularization, and dense classification layers enables the model to achieve strong performance in multiclass image classification tasks

5.3.2 Model Fitting Observation.

Observing the training and validation loss trajectory (Figure 13) of the CXC 101 model, the model clearly failed to reach a stable convergence. Initial observation indicates that both training and validation loss decreased through the second epoch and remain relatively stable until the fourth epoch. However, following the epochs exhibits a sharp and steady increase. This simultaneous rise in both curves is a strong indicator of model divergence rather than simple overfitting. Thus, the model is currently **unreliable and divergent**.

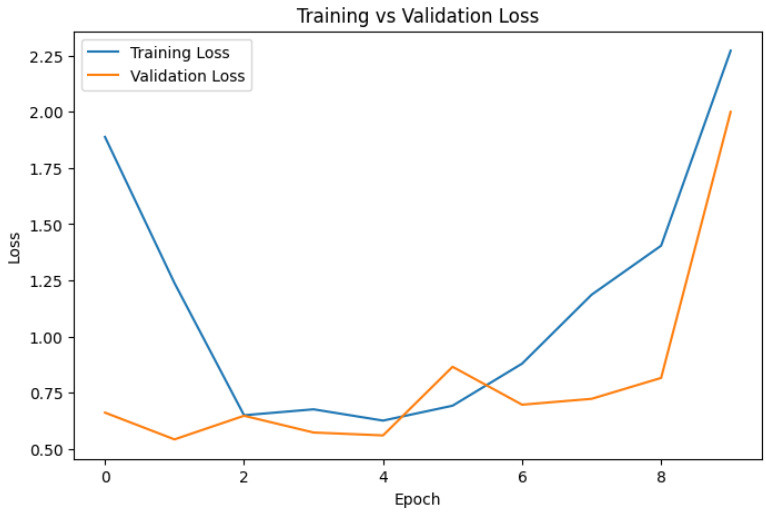


Figure 13: CXC 101 training loss graph

5.4 CXC 102

The CXC 102 model was developed as a second iteration following hyperparameter tuning with Keras Tuner. This model aimed to improve feature resolution by increasing the input image size to 384 × 384 × 3 and performing data augmentation. However, despite the higher resolution, the model faced challenges during training, specifically low accuracy as compared to the CXC 101, likely due to the increased complexity and class imbalance within the expanded dataset of 19,327 images.

The model achieved an overall validation accuracy of **84.36**%, which is quite lower than the initial results observed in CXC 101. This performance dip suggested that

simply increasing resolution without deeper architectural adjustments or more aggressive regularization might not be sufficient for this specific dataset.

5.4.1 Model Architecture and Layer-wise Parameter Description

The CXC 102 architecture maintains a similar structure to the CXC101 model while scaling up for higher-resolution inputs. The model architecture starts processing with Data augmentation layer, which performs random horizontal flip, rotation, zoom, translation, and contrast.

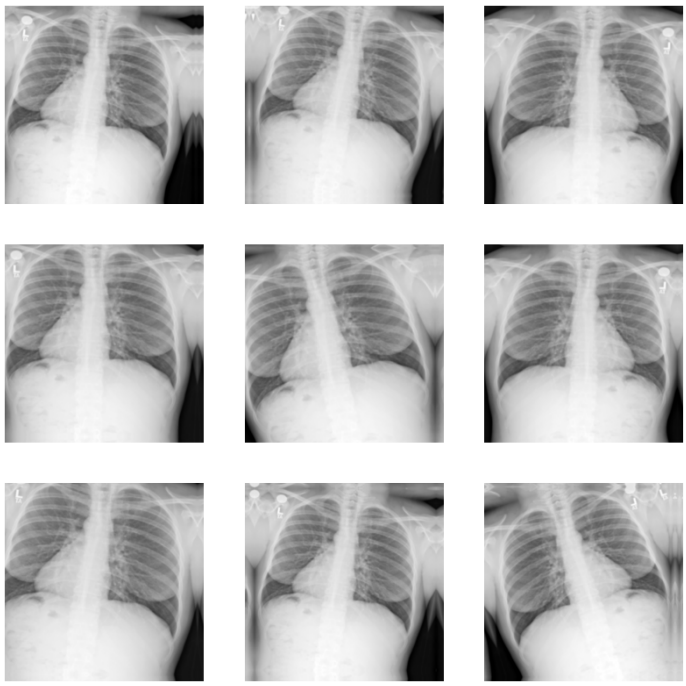


Figure 14: Image data augmentation

Then, a rescaling layer that normalizes the 384 × 384 × 3 input images, a significant increase from the 224 × 224 resolution used in the previous version. The feature extraction stage is composed of the same three primary convolutional blocks.

The first block uses a *Conv2D* layer with 32 filters and a *ReLU* activation to extract low-level features, resulting in 896 parameters. Followed by a *MaxPooling2D* layer to reduce dimensions.

The second block increases the depth with a Conv2D layer of 64 filters, contributing 18,496 parameters, while the third block further expands the feature space using 128 filters, adding 73,856 parameters. Consistent with the design of CXC 101, each convolutional layer in CXC 102 is paired with a max-pooling operation to maintain translation invariance and computational efficiency.

Once the spatial features are extracted, the resulting 3D tensors are transformed into a 1D vector through a *Flatten* layer. This vector feeds into the classification head, which is a dense layer of 128 units. This specific layer is the most parameter-intensive part of the model, containing 34,668,672 trainable parameters. To manage the risk of overfitting inherent in such a large parameter space, a dropout layer of 0.5 is applied, which randomly deactivates neurons during the training phase.

Finally, the model concludes with a four-unit dense output layer using a *softmax* activation function to categorize the images into one of the four diagnostic classes. In total, the CXC 102 model manages over 104 million parameters, with approximately 34.7 million of them being trainable, which is almost three times larger compared to CXC 101. In addition, the model learning rate is updated to 0.0001.

5.4.2 Model Fitting Observation.

Observing the training and validation accuracy plots (Figure 15) from the CXC102 model training history, it appears that the model achieves a significant accuracy of around 72% in the first epoch. However, in all subsequent epochs, the model only shows marginal improvements, ultimately reaching 75% training accuracy and 84% validation accuracy.

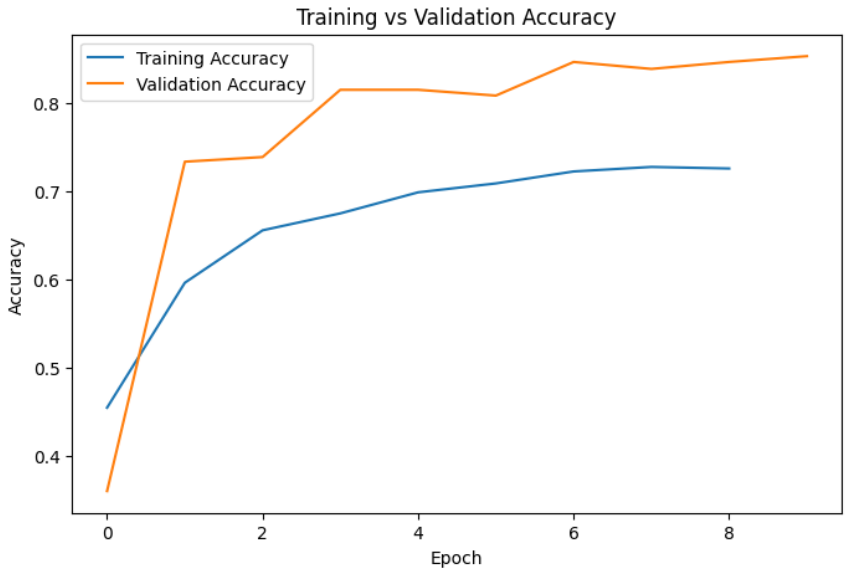


Figure 15: CXC 102 accuracy plot

However, the model's training loss graph of CXC102 shows that the loss steadily converges after the second iteration. During the first two epochs, the model experienced a sharp spike in both training and validation loss, likely due to the high learning rate relative to the increased complexity of the 384 × 384 input resolution. The model's training loss graph shows that the loss steadily converges after the second iteration. From the second through the ninth epoch, the training loss follows a consistent downward trajectory, falling from a peak of nearly 19.0 to approximately 6.0. Crucially, the validation loss mirrors this trend, showing that the model was effectively learning generalized features rather than simply memorizing the training data

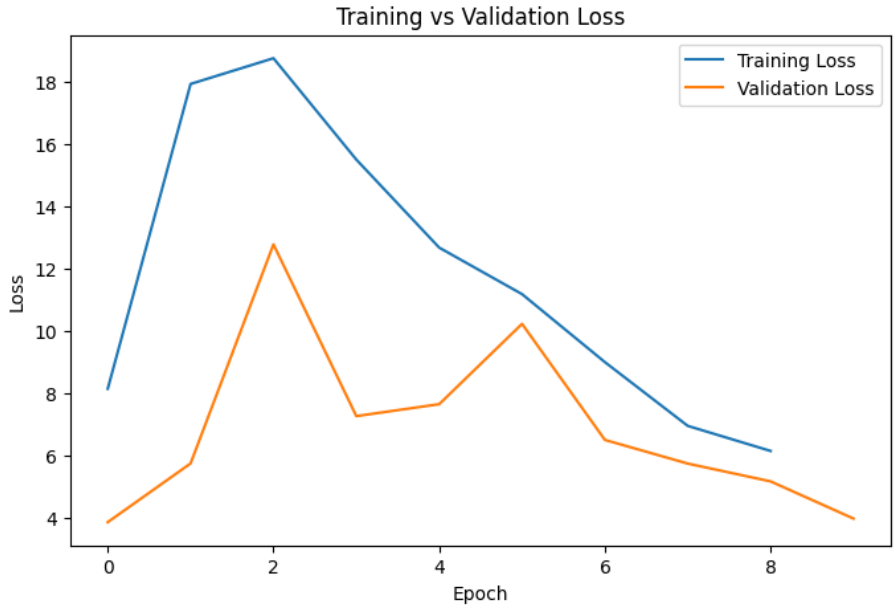


Figure 16: CXC 102 loss graph

5.5 CXC 103

The CXC 103 model has a higher accuracy of 98.59% training accuracy and 96.06% of validation accuracy. The model addresses the accuracy issue of the previous model (CXC 101 and CXC 102) via hyperparameter tuning and using batch normalization in each convolutional block. After each convolutional layer, batch normalizations are applied for better stability and to speed up convergence. The layer normalizes its output using the mean and standard deviation of the current batch of inputs. The observation image size resolution changes from CXC 102, leading to keeping the default image size to 224 × 224.

5.5.1 Model Architecture and Layer-wise Parameter Description

The CXC 103 model first applies the same synthetic data augmentation layer as in CXC 102, which artificially increases dataset diversity by generating modified versions of training images through transformations such as random flipping, rotation,

zooming, translation, and image contrast. This improves generalization and reduces overfitting.

Following augmentation, a Rescaling layer normalizes pixel intensities from the range (0-255) to (0-1), ensuring numerical stability and improving training efficiency. The input shape is defined as 224 × 224 × 3

The first feature extraction stage consists of a *Conv2D* layer with 32 filters of size 3 × 3 and *ReLU* activation, which extracts low-level image features such as edges, textures, and contours. A *Batch Normalization* layer follows to stabilize training and normalize activations, while *MaxPooling* (2 × 2) reduces dimensions and computational complexity.

The second convolutional block increases the number of filters to 64, enabling the network to learn more detailed and intermediate visual patterns within chest X-rays. This block also includes *Batch Normalization* and 2 × 2 *Max Pooling* to improve training stability and reduce feature map size. The third convolutional block further increases feature extraction capacity using 192 filters, allowing the model to learn higher-level and more complex anatomical patterns relevant for disease classification. Similar to earlier blocks, this stage applies *Batch Normalization* followed by *Max Pooling.*

After feature extraction, the multidimensional feature maps are converted into a one-dimensional vector using a Flatten layer, preparing the extracted features for classification. The classification stage begins with a Dense layer containing 128 neurons and *ReLU* activation, which learns discriminative combinations of extracted features for classification. Another *Batch Normalization* layer is applied to improve convergence and training stability. A Dropout layer with a rate of 0.5 randomly deactivates 50% of neurons during training to reduce overfitting and improve model generalization.

Finally, the output layer contains 4 neurons with *Softmax* activation, producing probability scores across the four target classes. The predicted class corresponds to the highest probability value. The model has a total of 50,235,918 parameters, were 16,745,028 are trainable parameters.

5.5.2 Model Fitting Observation.

The model shows solid optimization over the 10 epochs (Figure 17). Training accuracy steadily increases from 88.65% to 98.59%, while training loss consistently drops from 0.3522 to 0.0414, showing that the model is effectively learning from the training dataset. The validation metrics generally improve alongside the training metrics, reaching a peak validation accuracy of 96.98% and a low validation loss of 0.0800 in Epoch 8. This indicates strong generalization capabilities overall.

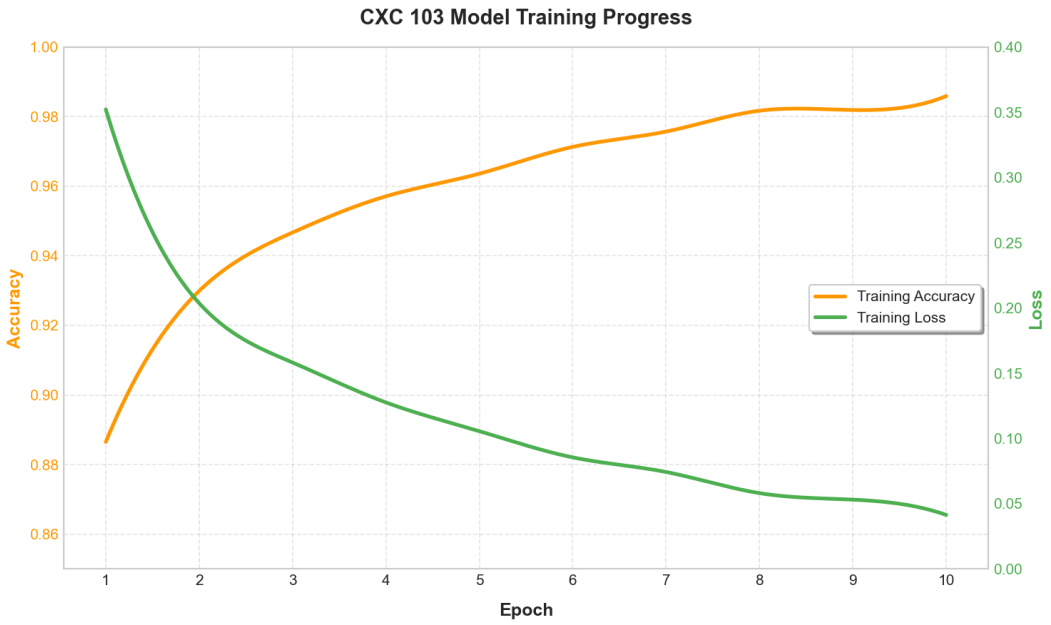


Figure 17: CXC 103 training accuracy v/s loss

5.6 CXC 201 (Inception V3)

The CXC 201 model was developed using the InceptionV3 transfer learning architecture, a deep learning model that was originally trained on the ImageNet

dataset. Unlike the custom CNN models used in the CXC 100-series, this model benefits from previously learned visual features, allowing it to identify meaningful patterns in chest X-ray images more effectively. InceptionV3 is designed to balance high classification performance with computational efficiency through its unique inception modules, which process information at multiple scales simultaneously. The model achieved a training accuracy of 90.11% and a validation accuracy of 91.20%, demonstrating its ability to learn relevant disease-related features and generalize well to unseen data

5.6.1 Model Architecture and Layer-wise Parameter Description

The CXC 201 architecture begins with an input layer that accepts chest X-ray images resized to 224 × 224 × 3 pixels. A preprocessing (Lambda) layer is then applied to prepare the images before they are passed to the main feature extraction network. The model uses InceptionV3's *preprocess_input* function to prepare the images in a way that matches the requirements of the pre-trained network

At the core of the model is the InceptionV3 backbone, which uses pre-trained ImageNet weights and excludes the original classification layer. This component is responsible for extracting important visual features from chest X-ray images, ranging from simple edges and textures to more complex anatomical structures and disease-related patterns. The InceptionV3 network generates feature maps of size 5 × 5 × 2048 and contains 21,802,784 parameters. Since the model utilizes transfer learning, these pretrained parameters are largely kept frozen during training, allowing the network to retain its previously learned visual knowledge.

The extracted feature maps are then passed through a Global Average Pooling 2D layer, which converts them into a compact 2048-dimensional feature vector. This process reduces the complexity of the model while preserving the most relevant information from the image.

Then, the feature vector is fed into a Dense layer with 256 neurons, containing 524,544 parameters. This layer learns patterns that are specific to chest X-ray classification and helps distinguish between the four target classes. To improve training stability and ensure consistent learning, a Batch Normalization layer with 1,024 parameters is applied after the dense layer.

A Dropout layer is then introduced to reduce overfitting by randomly disabling a portion of neurons during training. So, the model learns more robust features and improves its ability to generalize to new data. Finally, a Dense output layer with 4 neurons and a Softmax activation function generates probability scores for each class and produces the final prediction. This output layer contains 1,028 parameters.

Overall, the CXC 201 model contains 22,329,380 parameters, of which 526,084 are trainable, and 21,803,296 are non-trainable. By combining the powerful feature extraction capabilities of InceptionV3 with a lightweight custom classification head, the model achieves high classification performance while keeping the number of trainable parameters relatively low. This makes the architecture both efficient and effective for automated chest X-ray disease classification.

5.6.2 Model Fitting Observation.

The CXC 201 model showed consistent learning throughout the training process. At the beginning of training, the model achieved a training accuracy of 84.33% and a validation accuracy of 88.04%. As the epochs progressed, both training and validation performance improved, while the loss values gradually decreased. This shows that the model was able to learn useful patterns from the chest X-ray images and improve its classification capability over time.

By the final epoch, the model achieved a training accuracy of 90.11% with a training loss of 0.2457, and a validation accuracy of 91.20% with a validation loss of 0.2329. The close agreement between the final training and validation accuracies suggests that

the model maintained consistent performance on both the training and validation datasets, demonstrating effective learning throughout the training process.

5.7 CXC 202 (VGG-16)

The CXC 202 model was developed as an improved version of the CXC 201 model, with the main goal of achieving a higher accuracy and recall rate. To ensure a fair comparison between the two models, the overall architecture remained the same, allowing the experiment to focus on the impact of changing the feature extraction model. The model uses VGG16 for

5.7.1 Model Architecture and Layer-wise Parameter Description

The main difference in CXC 202 is the use of VGG16 as a pre-trained base model. Instead of training the feature extraction layers from scratch, the model takes advantage of VGG16's pre-trained knowledge, which was originally trained on a large image dataset. This helps the model recognize useful image patterns and features more effectively. In addition, the model uses VGG16's *preprocess_input* function to prepare the images in a way that matches the requirements of the pre-trained network.

The model accepts input images with dimensions of 224 × 224 × 3, which are first passed through the VGG16 feature extractor. The extracted features are then processed using a Global Average Pooling layer, followed by a Dense layer with 256 neurons to learn higher-level patterns. To improve training stability, Batch Normalization is applied, while a Dropout layer helps reduce overfitting and improve the model's ability to generalize to unseen data. Finally, the model produces predictions across four output classes. Overall, the CXC 202 model contains 15.1 million parameters, with only a small portion being trainable, as most of the VGG16 layers were frozen during training.

The experiment produced promising results, with the CXC 202 model achieving a validation accuracy of 94.88%. By replacing the original base model with VGG16, the

experiment aimed to improve recall performance while maintaining the strengths of the CXC 201 architecture. The results suggest that using transfer learning with VGG16 contributed positively to the model's performance and provided a strong foundation for image classification.

5.7.2 Model Fitting Observation.

The VGG16-based CXC 202 transfer learning model was trained over 10 epochs and demonstrated a stable learning process. Training accuracy steadily increased from 72.47% to 90.96%, while validation accuracy improved from 90.41% to a peak of 94.88%, showing strong and consistent performance throughout training.

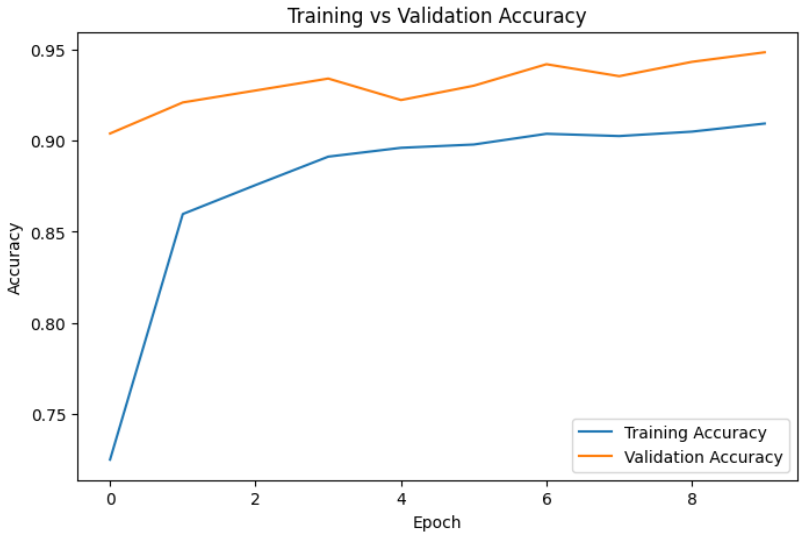


Figure 18: CXC 202 fitting accuracy plot

The loss metrics reflected this improvement, with training loss decreasing from 0.7763 to 0.2431 by the final epoch. Similarly, validation loss declined from 0.3131 to a minimum of 0.1528, with only minor fluctuations during training. The close alignment between training and validation performance indicates that the model maintained stability and achieved reliable generalization on unseen chest X-ray data.

5.8 CXC 203 (ResNet50)

The CXC 203 model was developed using the ResNet50 transfer learning architecture and represents one of the most advanced models evaluated in this project. ResNet50 introduces residual learning through shortcut connections, allowing information to flow directly across layers and helping to overcome the vanishing gradient problem commonly encountered in deep neural networks.

5.8.1 Model Architecture and Layer-wise Parameter Description

The ResetNet50 pre-trained model by default has 25.6 million parameters, and with 107 feature depth layers, yet it maintains a compact model size of just 98 MB (Mega Byte). In terms of performance, it achieves a 74.9% Top-1 accuracy and a 92.1% Top-5 accuracy, successfully outperforming both VGG16 and VGG19 while using roughly 80% fewer parameters. Furthermore, ResNet50 demonstrates excellent optimization for speed, clocking the fastest CPU inference time at 58.2 ms per step and a highly competitive GPU inference time of 4.6 ms per step [35].

The model begins with a preprocessing stage that uses a non-trainable Lambda layer (see Figure 19) to accept input images of size 224 × 224 × 3. This layer applies the ResNet50 *preprocess_input* function, which prepares the raw image data by converting images from RGB (Red Green Blue) to BGR (Blue Green Red) and adjusting pixel values based on ImageNet statistics. Since this layer only transforms the input data, it does not introduce any additional trainable parameters.

After preprocessing, the images are passed into the main feature extraction component, which uses a pre-trained ResNet50 model. This backbone generates a 7 × 7 × 2048 feature map, capturing important visual patterns from the input images. The backbone contains 23,587,712 non-trainable parameters, meaning the learned ImageNet features are retained and not updated during training.

To convert these extracted spatial features into a format suitable for classification, a *GlobalAveragePooling2D* layer is used. This layer averages each feature map and reduces the output to a 2048-dimensional vector, thereby lowering model complexity and reducing the likelihood of overfitting.

The classification stage then begins with a Dense layer containing 256 neurons, which learns meaningful combinations of the extracted features for prediction. This layer contributes 524,544 trainable parameters. Next, a *BatchNormalization* layer with 1,024 parameters is added to improve training stability, normalize activations, and support faster convergence.

To further improve generalization, a dropout layer is applied, randomly disabling some neurons during training to reduce overfitting. Finally, the model ends with a Dense output layer of 4 neurons using Softmax activation, which produces probability scores for the four target classes.

Overall, the architecture contains 25,166,478 parameters, of which 526,084 are trainable, and 23,588,224 are non-trainable, allowing the model to benefit from powerful pre-trained features while only fine-tuning a relatively small classification component.

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
batch_normalization (BatchNormalization)	(None, 256)	1,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 4)	1,028

Figure 19: CXC 203 model layers.

5.8.2 Model Fitting Observation.

The model was trained over 10 epochs, showing a smooth and stable learning process throughout training. Based on the training plots, the model followed a balanced learning pattern with no clear signs of overfitting or underfitting.

The training loss started at 0.3541 and steadily decreased to 0.0993 by the final epoch, indicating continuous improvement in the model's predictions. Similarly, the validation loss reduced from 0.1551 to 0.0551, suggesting that the model generalized well to unseen data.

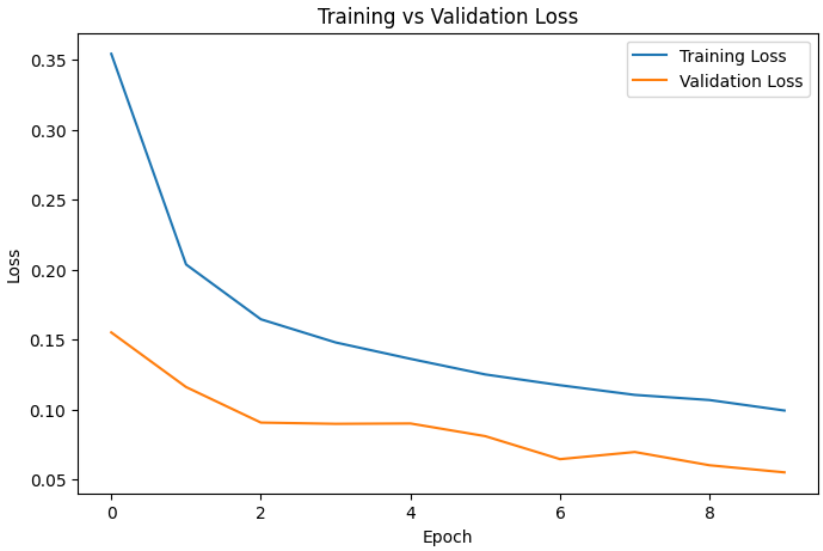


Figure 20: CXC 203 training loss over 10 epochs

This improvement is also reflected in the accuracy scores. The training accuracy gradually increased from 87.67% to 96.35%, while the validation accuracy improved from 94.48% to an impressive 98.03% in Epoch 10. Since the validation performance remained slightly higher than the training performance, it suggests that the dropout layer effectively reduced overfitting, helping the model learn general patterns rather than simply memorizing the training data.

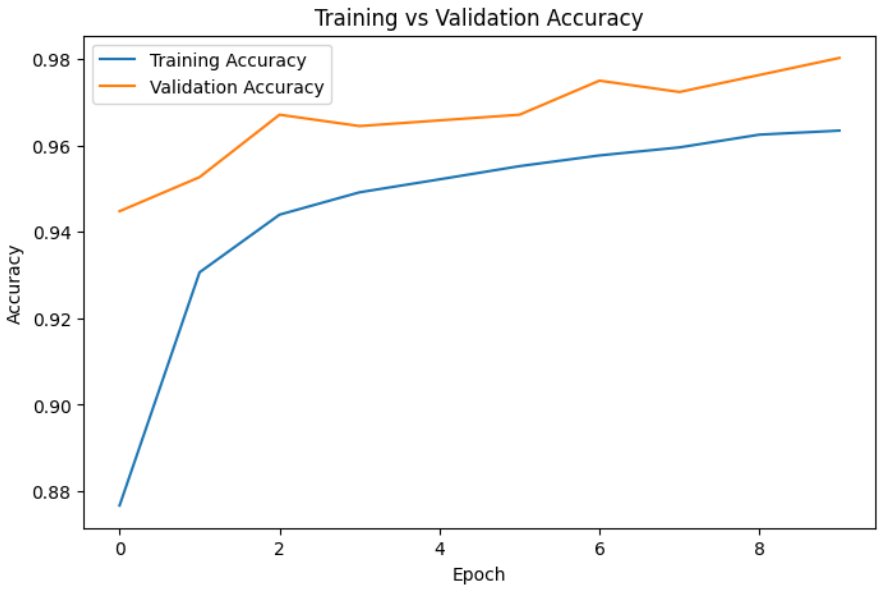


Figure 21: CXC 203 training accuracy over 10 epochs

Chapter 6. Testing and Evaluation

6.1 Evaluation Criteria

The models must achieve an accuracy of at least 90%, accounting for a 5% margin of error in theoretical medical experimentation. However, accuracy is not the sole evaluation metric for each model. According to the study, the primary objective is to develop a model capable of accurately identifying pneumonia and tuberculosis. If the model fails this criterion and misclassifies a disease as **"Normal",** the patient may be denied proper treatment or further diagnostic evaluation. Given that these are critical, life-altering decisions, the evaluation scale focuses on fine-tuning the model to minimize false negatives and incorrect predictions.

6.2 CXC Model Evaluation Table

The evaluation table shows clear variation in performance across the tested models.

MODEL NAME	Test Accuracy	Validation Accuracy	Satisfy Medical Error threshold
CXC 101	89.19%	88.04%	LOW
CXC 102	83.56%	85.28%	LOW
CXC 103	96.07%	96.06%	MEDIUM
CXC 201	89.46%	91.20%	LOW
CXC 202	92.53%	94.88%	MEDIUM
CXC 203	97.25%	98.03%	HIGH

Table 2: Model Comparison Table

6.3 CXC 101 Model Evaluation

The evaluation of the validation dataset is represented on a *Receiver Operating Characteristic* (ROC) curve and *Area Under the Curve* (AUC) (Figure 22). The model shows a near-perfect classification for the "UNKNOWN" class and high discriminatory power for *Pneumonia* (0.99 AUC), indicating a strong true-positive rate with minimal false positives across all thresholds. While the NORMAL and TB classes also demonstrate high AUC values of 0.97 and 0.96, respectively, the slight inward curvature of their plots suggests a more complex decision boundary between healthy lung and TB.

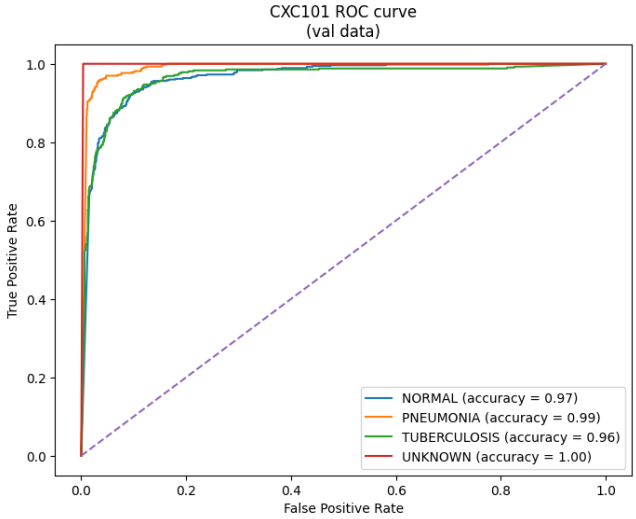


Figure 22: CXC 101 ROC curve

Figure 23, the confusion matrix on validation data, highlights a critical sensitivity for the TB class. Although the model maintains a high overall accuracy of 88%, the recall for TB is notably lower at 0.77, as 38 actual TB cases were misidentified as Normal. This significant false-negative rate suggests that the CNN often fails to capture the distinct pathological markers of TB, leading to a conservative bias toward the Normal class. Conversely, the Pneumonia class exhibits high reliability with a 0.94 F1-score and 0.95 recall. The uknown class remains the most stable, achieving a 1.00 recall.

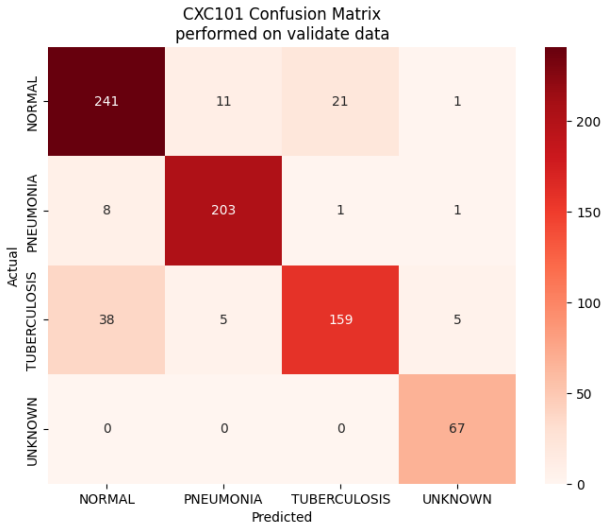


Figure 23: CXC 101 Confusion Matrix

6.4 CXC 102 Model Evaluation

The CXC 102 model was evaluated on the test dataset, and the classification report (Figure 24) shows that the model achieved an overall accuracy of **84**%. However, the recall rate for TB (0.68) and normal healthy (0.85) lung chest X-ray classes is significantly lower.

	precision	recall	f1-score	support
NORMAL	0.77	0.85	0.81	548
PNEUMONIA	0.85	0.93	0.89	427
TUBERCULOSIS	0.87	0.68	0.77	417
UNKNOWN	0.98	0.97	0.97	135
accuracy			0.84	1527
macro avg	0.87	0.86	0.86	1527
weighted avg	0.84	0.84	0.83	1527

Figure 24: CXC 102 classification report on test dataset

Looking at the confusion matrix (Figure 25) on the test dataset, almost 112 TB images are misclassified as normal images. This is a critical false-negative error for TB, which is highly significant in a clinical context because a serious illness goes undetected. Plus, 51 normal cases were falsely predicted as Pneumonia, and 33

NORMAL cases were falsely predicted as TB. Thus, the model fails to achieve the required evaluation criteria, and due to ethical and other reasons, it is not suitable for clinical purposes.

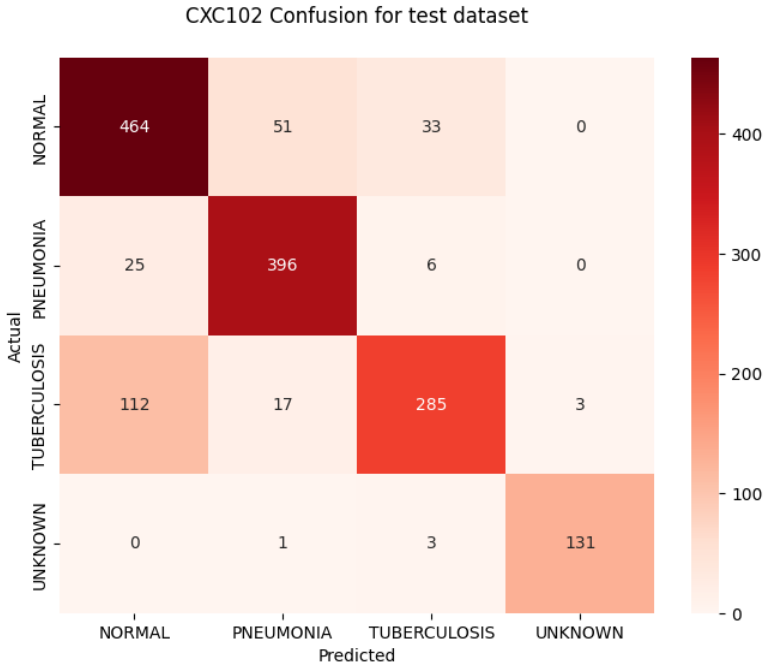


Figure 25: CXC 102 Confusion Matrix

6.5 CXC 103 Model Evaluation

The CXC 103 model has the highest accuracy matrix among the custom-built CNN models, as compared to all the models, successfully resolving the performance bottlenecks and divergence issues observed in both the baseline CXC 101 and intermediate CXC 102 architectures. When evaluated against the primary diagnostic criteria, the CXC 103 model established the highest metric profile among all tested configurations, securing an overall classification accuracy of 96% on both the test and validation datasets.

This performance satisfies the target model accuracy threshold of 90%, which is required to comfortably accommodate. But due to the clinical margin of error in experimental medical requirements, it might fail as the recall rate of the TB is 0.89. Analysis of the model's confusion matrix (Figure 26) and associated classification

reports demonstrates an exceptional capacity to distinguish between most of the TB and Pneumonia while avoiding severe false-negative classifications. Unlike its predecessors, which exhibited a conservative bias that frequently misclassified hazardous TB cases as normal, the CXC 103 architecture demonstrates a high and well-balanced recall rate across all target classes.

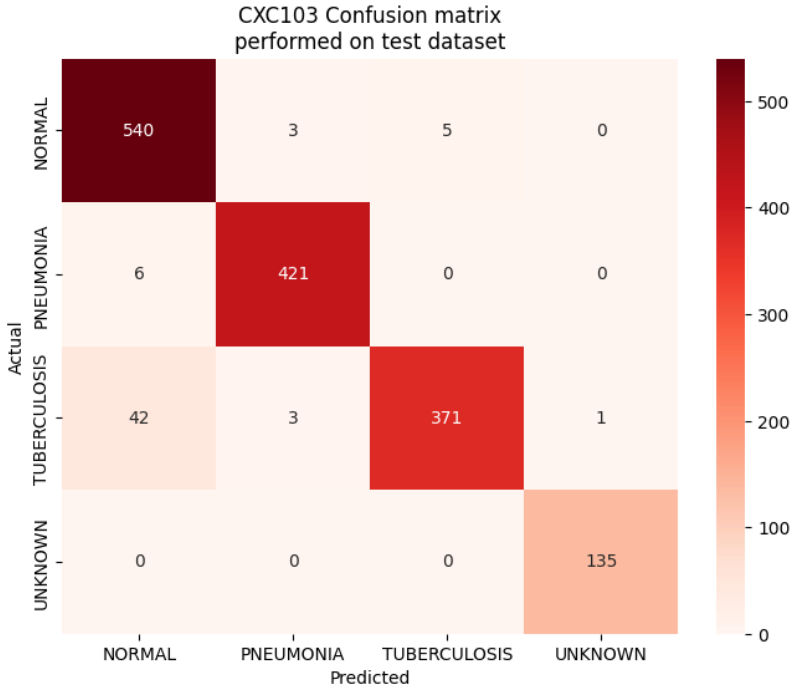


Figure 26: CXC 103 confusion matrix

Furthermore, the model's Receiver Operating Characteristic (ROC) curve (Figure 27) displays a near-ideal trajectory, with the area under the curve (AUC) moving aggressively into the upper-left quadrant to yield a highly reliable discriminatory power. This strong statistical distribution confirms that the CNN can reliably distinguish subtle variations in images, with minimal false-positive crossover. By maintaining a high F1-score and exceptional specificity across the main diagnostic classes, as well as the robust "Unknown" catch-all category, the CXC 103 model achieves the baseline validation necessary to prove its potential utility for real-world clinical screening software.

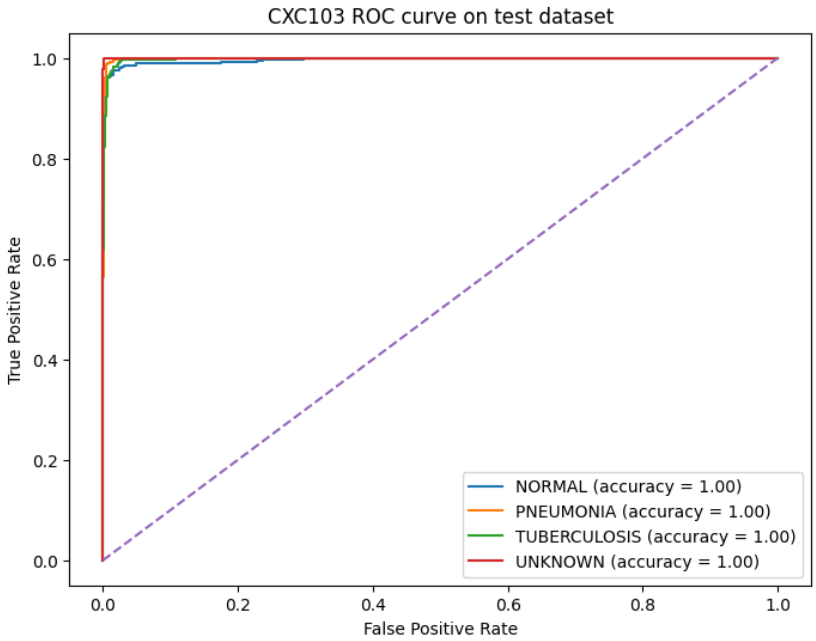


Figure 27: CXC 103 ROC Curve

On user testing, the CXC 103 model shows the best performance as compared to any other model. Special when testing with random images and images from the phone cameras. The model shows a balanced proportion when predicting the images.

6.6 CXC 201 Model Evaluation

The CXC 201 model achieved an overall test accuracy of 89%. However, when compared with the custom-developed CXC 103 model, CXC 201 achieved a relatively lower classification accuracy. Although the transfer learning approach using InceptionV3 provided effective feature extraction and produced reliable results, the custom CXC 103 architecture was able to achieve better overall performance on the same classification task. This suggests that the architecture and training strategy used in CXC 103 were more suited to the characteristics of the chest X-ray dataset.

The model performed particularly well in detecting Pneumonia, achieving a precision of 88% and a recall of 0.97, indicating that it successfully identified most pneumonia cases in the dataset. The model also showed excellent performance for the Unknown

class, correctly classifying all 135 samples and achieving 100% precision and 100% recall.

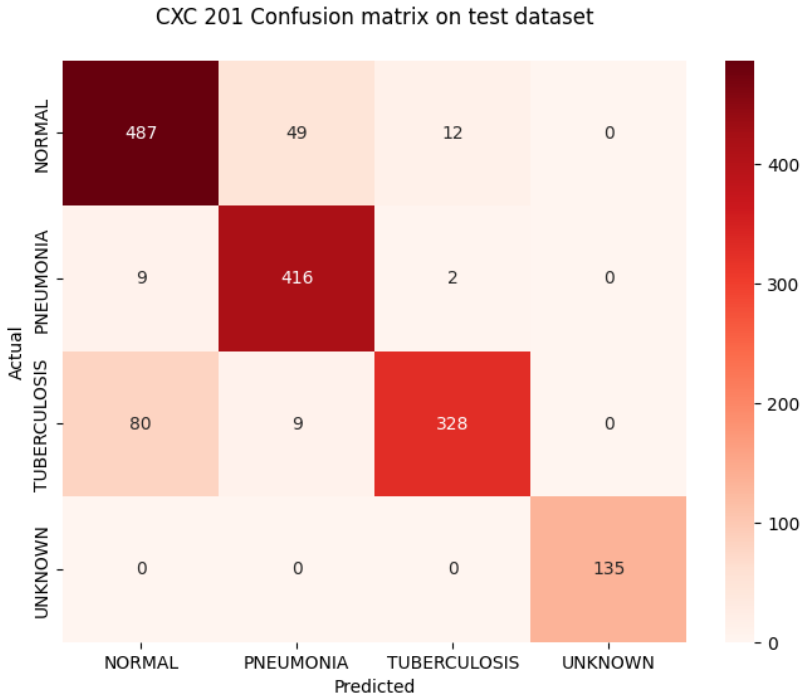


Figure 28: CXC 201 Confusion Matrix

However, the model obtained a recall of 79% for Tuberculosis, meaning that some TB cases were not correctly identified. Analysis of the confusion matrix (Figure 28) showed that some TB cases were classified as Normal or Pneumonia, which reduced the model's ability to detect all TB-positive patients.

6.7 CXC 202 Model Evaluation

The evaluation results for the CXC 202 model overall achieved 93% accuracy on a test dataset. It performed very well in detecting Pneumonia, with 94% precision and 98% recall, meaning it correctly identified most pneumonia cases. The model also classified all “Unknown” cases correctly.

However, performance was weaker for some important categories. Recall for Tuberculosis was 87% and for Normal cases was 93%, meaning some cases were misclassified. In particular, 50 TB cases were predicted as Normal and 6 as Pneumonia. In addition, 36 healthy cases were incorrectly classified as diseased. These results show that although the model performs well overall, it still misses some important cases, especially Tuberculosis. Because of this, it would need further improvement before being used in real clinical settings.

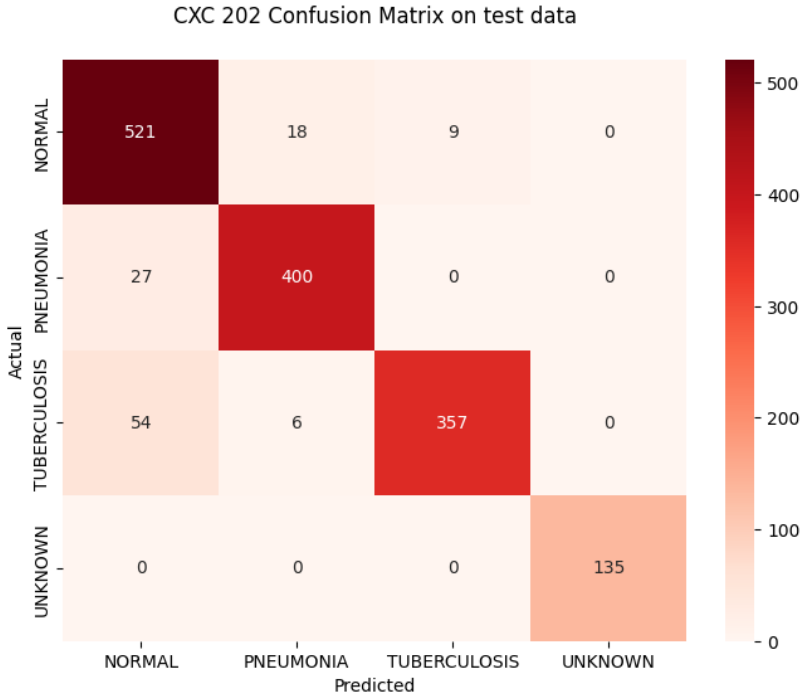


Figure 29: CXC 202 confusion matrix on test data

6.8 CXC 203 Model Evaluation

Based on the established evaluation criteria, the CXC 203 model successfully meets the required performance target by achieving an overall test accuracy of 97%, which is well above the minimum benchmark of 90%. Plus, in medical diagnosis, accuracy alone is not enough. Since incorrect predictions can directly affect patient health, the model must also be assessed on its ability to reduce false negatives, particularly cases where patients with pneumonia or tuberculosis are mistakenly classified as “Normal”.

The model performs particularly well in this area. For pneumonia detection, the results are highly reliable, with zero false negatives recorded. This means that no patient with pneumonia was incorrectly classified as healthy, reducing the risk of delayed treatment or missed diagnosis. For tuberculosis detection, the model also demonstrates strong performance, although there is a small limitation. Out of 417 confirmed tuberculosis cases, 16 were incorrectly classified as “Normal”. While this introduces a minor clinical concern, the number of missed cases remains relatively low compared to the total number of infections detected.

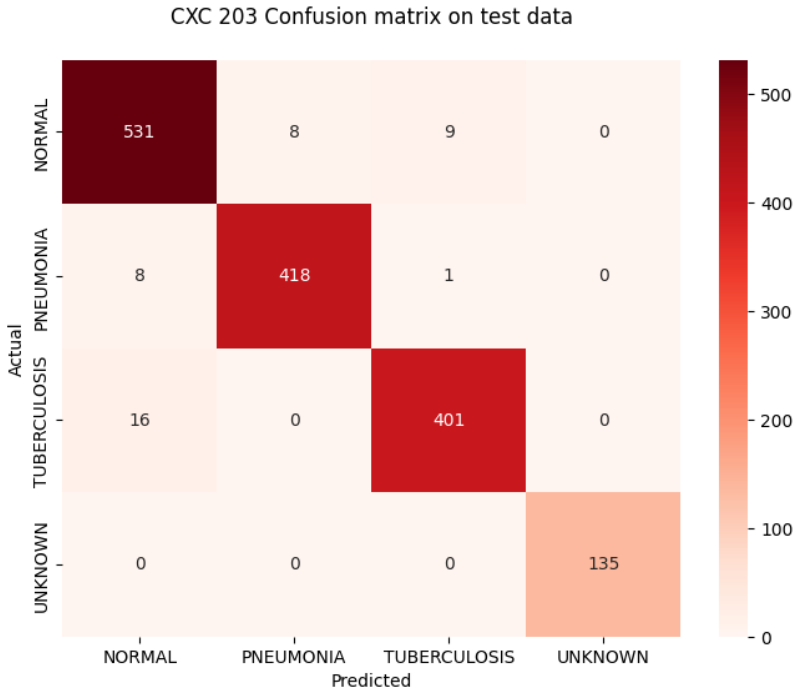


Figure 30: CXC 203 confusion matrix on test data

Overall, the CXC 203 model shows strong reliability as a diagnostic tool. By maintaining high accuracy and minimizing missed infections, particularly in serious respiratory diseases, the model demonstrates strong potential to support clinical decision-making and assist healthcare professionals in identifying patients who require further medical attention.

6.9 Application User Testing

The CXC Flutter application was tested across a variety of platforms and devices to ensure a responsive and user-friendly experience. During the initial testing phase, users reported several UI-related issues, including overflow errors on smaller mobile devices. These issues were identified and fixed, resulting in improved responsiveness and compatibility across different screen sizes.

Users also provided valuable feedback regarding model transparency and usability. Many requested additional documentation to better understand the purpose and differences between the available models. In response, detailed model information was added to the application. Another common suggestion was to display a clear indicator, like a badge, showing the currently selected default model, which was later implemented to improve navigation and user awareness.

Performance testing further revealed that the batch classification feature could become slow when processing large numbers of images. To improve efficiency, the prediction workflow was optimized by processing images in chunks during batch classification. This enhancement reduced prediction times and provided a smoother user experience when classifying multiple images.

The application was also evaluated by professional radiologists and medical practitioners. They provided positive feedback on the prediction results generated by the CXC 103 and CXC 203 models, noting that the classifications were generally accurate and clinically meaningful. Their feedback helped validate the effectiveness of the models and highlighted the potential of the CXC Flutter application as a supportive tool for chest X-ray analysis.

Chapter 7. Conclusions and Future Work

7.1 Conclusions

In Conclusion, the development of the Chest X-ray Classifier (CXC) system highlights how deep learning can move beyond academic research and become a practical tool for healthcare. With growing challenges in modern medicine, particularly the shortage of radiologists and delays in diagnosing serious lung conditions such as Pneumonia and tuberculosis, this project aimed to create a system that could support faster and more efficient diagnosis. By combining powerful neural network models with an easy-to-use, cross-platform Flutter application, the project successfully delivered a clinical assistant designed to improve triage processes and reduce pressure on healthcare professionals.

Testing different model architectures provided various important insights into the trade-off between high performance and practical usability in real-world settings. The custom models helped build a stronger understanding of feature extraction and model behaviour. Although the first version, CXC 101, did not achieve the level of accuracy needed for clinical reliability, continuous improvements, such as fine-tuning, led to the development of CXC 103. With the addition of techniques such as dropout and batch normalization, the model became significantly more robust. During testing, CXC 103 performed consistently well even in less controlled conditions, successfully analysing smartphone photographs of physical X-ray films despite lighting variations and image noise.

The use of pre-trained models proved to be the most effective strategy for achieving strong clinical performance. Among all implementations, the ResNet50-based CXC 203 model delivered the best results, reaching a validation accuracy of 98.03% and an overall test accuracy of 97%. Most importantly, it achieved a zero false-negative rate for Pneumonia, meaning no positive cases were missed during testing, an essential outcome for patient safety.

To make these models accessible in a real clinical setting, they were deployed through a Fast API backend and integrated into a user-friendly Flutter application. Features such as multi-model comparisons and batch processing through ZIP uploads make the system practical for day-to-day clinical use. Overall, the project demonstrates that combining advanced machine learning with thoughtful software design can create tools that meaningfully support healthcare professionals and improve diagnostic workflows.

7.2 Future Work and Enhancement

Although the CXC system successfully achieved its main objectives, medical image diagnostics continues to evolve, creating several opportunities for future improvement. One key area for development is the implementation of a model ensembling approach, where the strongest-performing CXC models work together to make predictions. By combining outputs from both custom CNN and transfer learning models through a weighted decision process, the system could improve diagnostic accuracy, reduce false positives, and provide more consistent predictions across all target conditions.

Another important enhancement is moving beyond full-image classification toward image segmentation and object detection. Future versions of the system could identify and highlight suspicious lung regions directly on the X-ray, helping medical professionals locate areas associated with Pneumonia or Tuberculosis more easily. Integrating advanced detection models such as YOLO could further support faster and more precise identification of abnormalities. In addition, an AI-powered text explanation feature could be introduced to generate simple clinical summaries, helping explain why a diagnosis was suggested and improving system transparency.

Future development will also aim to expand the system beyond its current three diagnostic classes by introducing additional chest X-ray diseases and respiratory conditions. Incorporating a wider range of abnormalities, such as lung cancer, pleural effusion, chronic obstructive pulmonary disease (COPD), and other thoracic

conditions, would improve the clinical usefulness of the system and allow it to support a broader range of diagnostic scenarios.

To transition the CXC system into a more production-ready medical application, improvements to the software infrastructure are also necessary. Future work should include stronger unit and integration testing across both the Flutter application and FastAPI backend to improve reliability. Automated CI/CD pipelines could streamline testing and deployment, while a secure feedback and retraining workflow would allow new clinical data to be continuously incorporated, helping the models improve over time.

References

[1] “Machine learning in healthcare: Uses, benefits and pioneers in the field,” EIT Health. Accessed: May 01, 2026. [Online]. Available: https://eithealth.eu/news-article/machine-learning-in-healthcare-uses-benefits-and-pioneers-in-the-field/

[2] “AI cuts time for radiologists reporting fractures on x-rays,” AuntMinnie. Accessed: May 24, 2026. [Online]. Available: https://www.auntminnie.com/clinical-news/digital-x-ray/article/15659139/ai-cuts-time-for-radiologists-reporting-fractures-on-xrays

[3] “What Is Tuberculosis?,” Cleveland Clinic. Accessed: May 02, 2026. [Online]. Available: https://my.clevelandclinic.org/health/diseases/11301-tuberculosis

[4] “Pneumonia,” HSE.ie. Accessed: May 02, 2026. [Online]. Available: https://www2.hse.ie/conditions/pneumonia/

[5] “Chest X-ray for tuberculosis (TB): What to expect, results, and more.” Accessed: May 03, 2026. [Online]. Available: https://www.medicalnewstoday.com/articles/tuberculosis-x-ray

[6] “Chest X-ray - Pulmonary disease.” Accessed: May 03, 2026. [Online]. Available: https://www.radiologymasterclass.co.uk/gallery/chest/pulmonary-disease/tuberculosis_tb

[7] “Spotting Pneumonia on X-Ray and What It Means - Centers Urgent Care.” Accessed: May 03, 2026. [Online]. Available: https://centersurgentcare.net/what-pneumonia-looks-like-on-x-ray/general/

[8] “Difference between Pneumonia and Tuberculosis X-ray,” Knya. Accessed: May 03, 2026. [Online]. Available: https://knyamed.com/blogs/difference-between/pneumonia-vs-tuberculosis-x-ray

[9] “Chest X-ray - Pulmonary disease.” Accessed: May 03, 2026. [Online]. Available: https://www.radiologymasterclass.co.uk/gallery/chest/pulmonary-disease/atypical_pneumonia

[10] “The Issues,” Every Breath Counts. Accessed: May 24, 2026. [Online]. Available: https://stoppneumonia.org/issues/

[11] “Tuberculosis (TB).” Accessed: May 24, 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/tuberculosis

[12] A. P. Brady *et al.*, “A project to define standards for radiology staffing, education and training across the European Union,” *Insights Imaging*, vol. 16, no. 1, p. 56, Mar. 2025, doi: 10.1186/s13244-025-01924-8.

[13] A. P. Brady *et al.*, “Guidelines and recommendations for radiologist staffing, education and training,” *Insights Imaging*, vol. 16, no. 1, p. 57, Mar. 2025, doi: 10.1186/s13244-025-01926-6.

[14] S. Iqbal, A. N. Qureshi, J. Li, and T. Mahmood, “On the Analyses of Medical Images Using Traditional Machine Learning Techniques and Convolutional Neural Networks,” *Arch Computat Methods Eng*, vol. 30, no. 5, pp. 3173–3233, Jun. 2023, doi: 10.1007/s11831-023-09899-9.

[15] P. Rajpurkar *et al.*, “CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning,” Dec. 25, 2017, *arXiv*: arXiv:1711.05225. doi: 10.48550/arXiv.1711.05225.

[16] “Frontiers | Edge AI for Healthcare Operations: From Real-Time Monitoring to Organizational Intelligence.” Accessed: May 26, 2026. [Online]. Available: https://www.frontiersin.org/research-topics/77562/edge-ai-for-healthcare-operations-from-real-time-monitoring-to-organizational-intelligence

[17] A. Rocha *et al.*, “Edge AI for Internet of Medical Things: A literature review,” *Computers and Electrical Engineering*, vol. 116, p. 109202, May 2024, doi: 10.1016/j.compeleceng.2024.109202.

[18] V. Bundele *et al.*, “Evaluating Self-Supervised Learning in Medical Imaging: A Benchmark for Robustness, Generalizability, and Multi-Domain Impact,” arXiv.org. Accessed: May 01, 2026. [Online]. Available: https://arxiv.org/abs/2412.19124v2

[19] H. Guo, S. A. Somayajula, R. Hosseini, and P. Xie, “Improving image classification of gastrointestinal endoscopy using curriculum self-supervised learning,” *Sci Rep*, vol. 14, no. 1, p. 6100, Mar. 2024, doi: 10.1038/s41598-024-53955-8.

[20] H. Laçi, K. Sevrani, and S. Iqbal, “Deep learning approaches for classification tasks in medical X-ray, MRI, and ultrasound images: a scoping review,” *BMC Med Imaging*, vol. 25, no. 1, p. 156, May 2025, doi: 10.1186/s12880-025-01701-5.

[21] J. W. Kim, A. U. Khan, and I. Banerjee, “Systematic Review of Hybrid Vision Transformer Architectures for Radiological Image Analysis,” *J Digit Imaging. Inform. med.*, vol. 38, no. 5, pp. 3248–3262, Oct. 2025, doi: 10.1007/s10278-024-01322-4.

[22] M. Ghassemi, T. Naumann, P. Schulam, A. L. Beam, I. Y. Chen, and R. Ranganath, “A Review of Challenges and Opportunities in Machine Learning for Health,” *AMIA Jt Summits Transl Sci Proc*, vol. 2020, pp. 191–200, May 2020.

[23] A. Kanapari, G. Lorenzoni, H. Ocagli, and D. Gregori, “Current applications and future challenges of machine learning and artificial intelligence in clinical trials: A scoping review,” *Digit Health*, vol. 11, p. 20552076251393272, Dec. 2025, doi: 10.1177/20552076251393272.

[24] S. Srivastava, E. Ghosh, A. Kumar, P. Chahar, A. Utkarsh, and R. Mishra, “Multi-class deep learning architecture for COVID-19, tuberculosis, and pneumonia classification using chest X-ray images,” *J Med Imaging Radiat Sci*, vol. 56, no. 6, p. 102115, Dec. 2025, doi: 10.1016/j.jmir.2025.102115.

[25] K. Team, “Keras documentation: Keras Applications.” Accessed: May 01, 2026. [Online]. Available: https://keras.io/api/applications/

[26] G. Kourounis, A. A. Elmahmudi, B. Thomson, J. Hunter, H. Ugail, and C. Wilson, “Computer image analysis with artificial intelligence: a practical introduction to convolutional neural networks for medical professionals,” *Postgrad. Med. J.*, vol. 99, no. 1178, pp. 1287–1294, Dec. 2023, doi: 10.1093/postmj/qgad095.

[27] “CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning.” Accessed: May 31, 2026. [Online]. Available: https://stanfordmlgroup.github.io/projects/chexnet/

[28] “The 1-Neuron Network: Logistic Regression,” The Data Frog. Accessed: May 29, 2026. [Online]. Available: https://thedatafrog.com/en/articles/logistic-regression/

[29] Jorgecardete, “Convolutional Neural Networks: A Comprehensive Guide,” The Deep Hub. Accessed: May 29, 2026. [Online]. Available: https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175

[30] “Activation functions in Neural Networks,” GeeksforGeeks. Accessed: May 29, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/

[31] M. Paks, “Lobar pneumonia | Radiology Reference Article | Radiopaedia.org,” Radiopaedia. Accessed: May 03, 2026. [Online]. Available: https://radiopaedia.org/articles/lobar-pneumonia

[32] V. Jain, R. Vashisht, G. Yilmaz, and A. Bhardwaj, “Pneumonia Pathology,” in *StatPearls*, Treasure Island (FL): StatPearls Publishing, 2026. Accessed: May 03, 2026. [Online]. Available: http://www.ncbi.nlm.nih.gov/books/NBK526116/

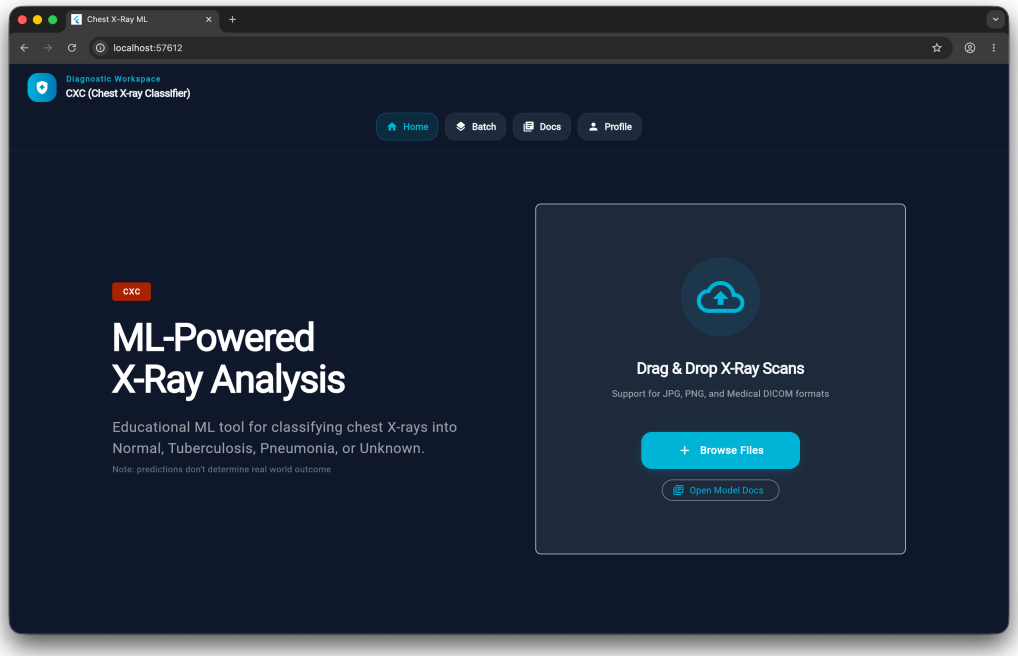
[33] F. Gaillard, “Tuberculosis (pulmonary manifestations) | Radiology Reference Article | Radiopaedia.org,” Radiopaedia. Accessed: May 26, 2026. [Online]. Available: https://radiopaedia.org/articles/tuberculosis-pulmonary-manifestations-1

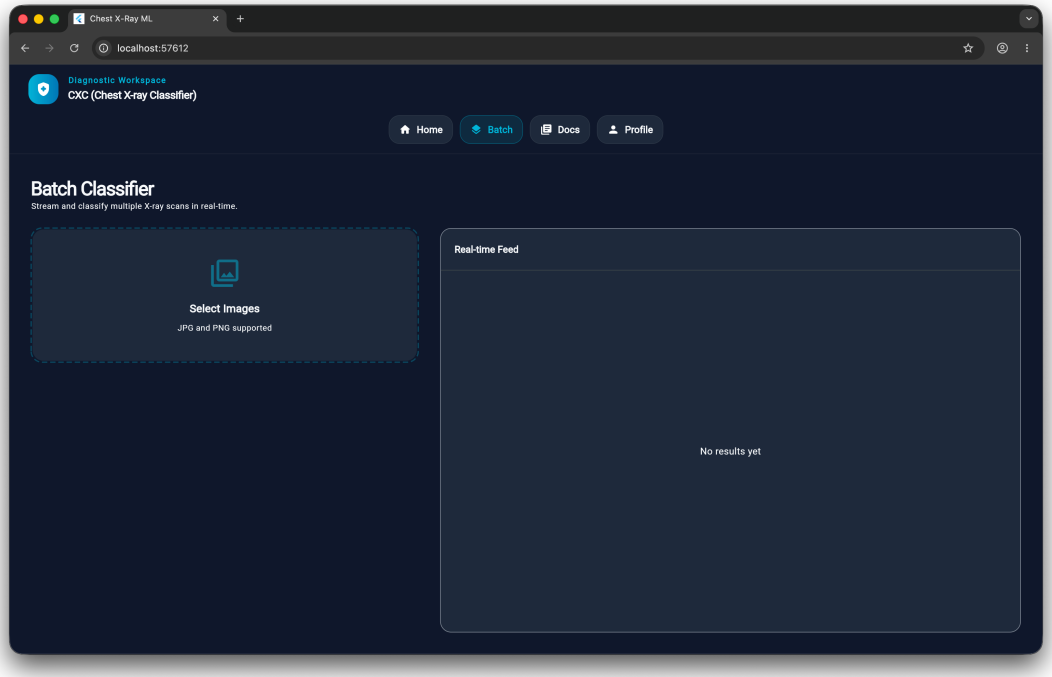
[34] “Chest X-ray - Pulmonary disease.” Accessed: May 26, 2026. [Online]. Available: https://www.radiologymasterclass.co.uk/gallery/chest/pulmonary-disease/tuberculosis_tb

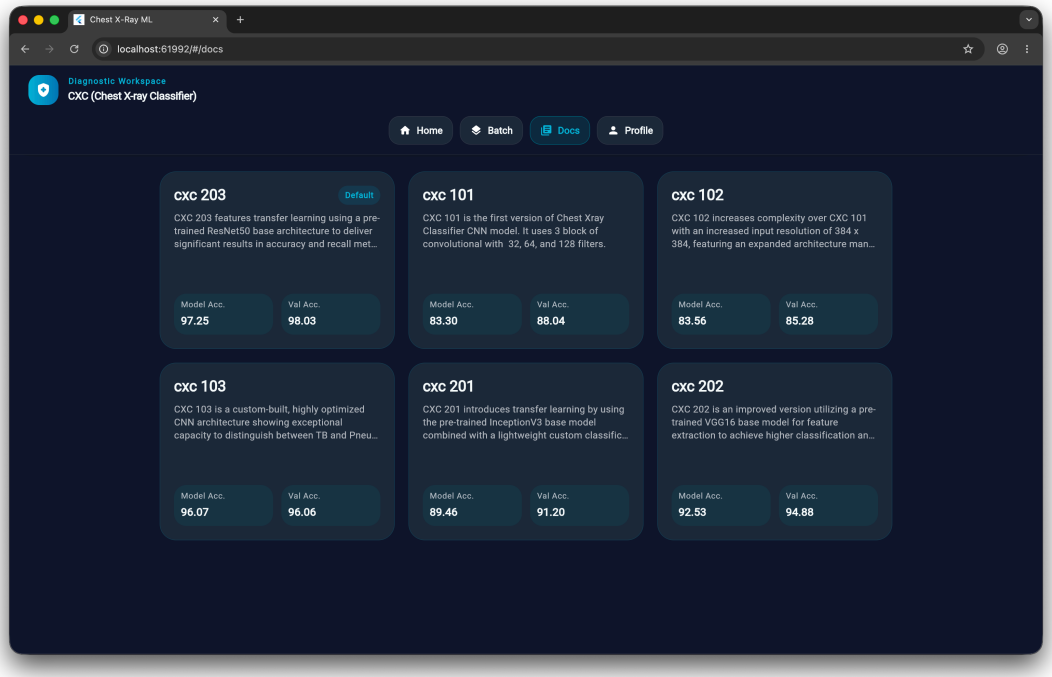
[35] K. Team, “Keras documentation: Keras Applications.” Accessed: May 28, 2026. [Online]. Available: https://keras.io/api/applications/

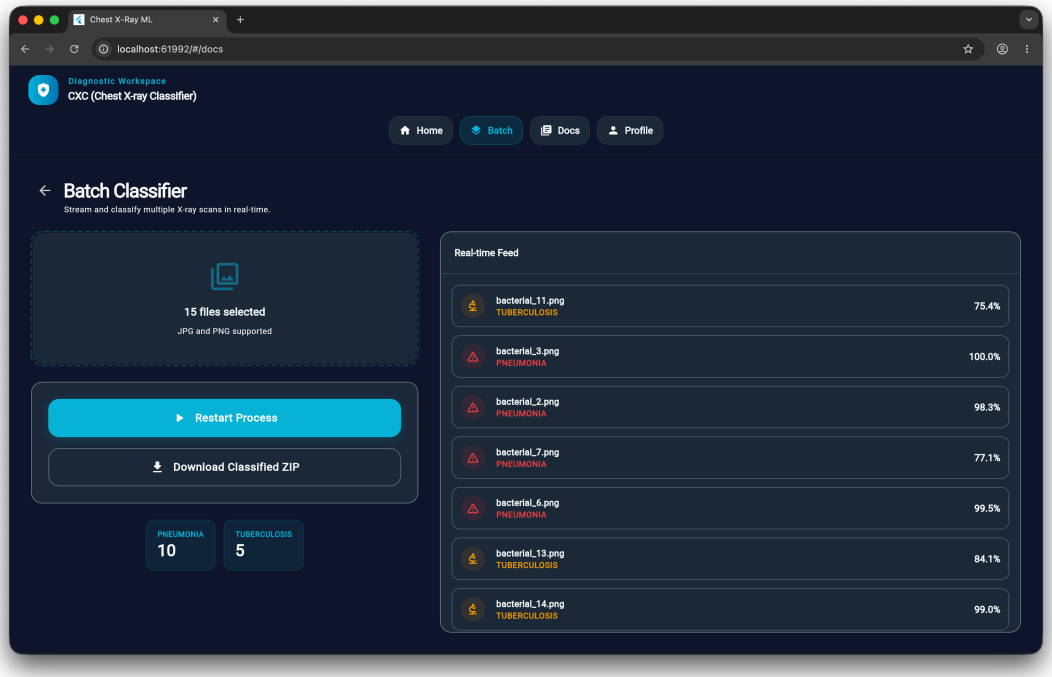
Appendix I

CXC application UI, web view






Chest X-Ray ML
localhost:61992/#/docs
Diagnostic Workspace
CXC (Chest X-ray Classifier)
Home
Batch
Docs
Profile
cxc 203
Default
CXC 203 features transfer learning using a pre-trained ResNet50 base architecture to deliver significant results in accuracy and recall met...
Model Acc.
97.25
Val Acc.
98.03
cxc 101
CXC 101 is the first version of Chest Xray Classifier CNN model. It uses 3 block of convolutional with 32, 64, and 128 filters.
Model Acc.
83.30
Val Acc.
88.04
cxc 102
CXC 102 increases complexity over CXC 101 with an increased input resolution of 384 x 384, featuring an expanded architecture man...
Model Acc.
83.56
Val Acc.
85.28
cxc 103
CXC 103 is a custom-built, highly optimized CNN architecture showing exceptional capacity to distinguish between TB and Pneu...
Model Acc.
96.07
Val Acc.
96.06
cxc 201
CXC 201 introduces transfer learning by using the pre-trained InceptionV3 base model combined with a lightweight custom classific...
Model Acc.
89.46
Val Acc.
91.20
cxc 202
CXC 202 is an improved version utilizing a pre-trained VGG16 base model for feature extraction to achieve higher classification an...
Model Acc.
92.53
Val Acc.
94.88


Chest X-Ray ML
localhost:61992/#/docs
Diagnostic Workspace
CXC (Chest X-ray Classifier)
Home
Batch
Docs
Profile
Batch Classifier
Stream and classify multiple X-ray scans in real-time.
15 files selected
JPG and PNG supported
Restart Process
Download Classified ZIP
PNEUMONIA
10
TUBERCULOSIS
5
Real-time Feed
bacterial_11.png
TUBERCULOSIS
75.4%
bacterial_3.png
PNEUMONIA
100.0%
bacterial_2.png
PNEUMONIA
98.3%
bacterial_7.png
PNEUMONIA
77.1%
bacterial_6.png
PNEUMONIA
99.5%
bacterial_13.png
TUBERCULOSIS
84.1%
bacterial_14.png
TUBERCULOSIS
99.0%

Appendix II

CXC application mobile view showing light and dark views.

